

Lava: a Translation from Liquid Haskell to Rocq

ANONYMOUS AUTHOR(S)

Large scale formal program verification can be carried out in a wide spectrum of foundational systems. At one end are refinement type systems, such as Liquid Haskell, which provide lightweight, highly automated verification integrated into mainstream programming languages. Their automation—powered by SMT solvers—makes them effective and practical, but also introduces a semantic gap between program behavior and solver reasoning that is not fully formalized. At the other end are interactive theorem provers based on type theory, such as Rocq, which come with small trusted kernels and strong logical foundations. These systems are flexible and trustworthy, but typically require more manual proof effort and expertise.

We bridge this gap by connecting SMT-based refinement type checking with type theory. We present the design, implementation, and metatheory of Lava, a translation from a substantial fragment of the refinement calculus underlying Liquid Haskell to programs in the type theoretic proof assistant Rocq.

Lava serves three purposes: it is a prototyping framework that enables program development in the practical front end of Liquid Haskell and attempts to generate sound certificates in Rocq; it provides foundations for Liquid Haskell in terms of Rocq’s well understood type theory; and it allows users to manually intervene when SMT solvers do not find proofs. Our formal development and implementation of Lava sets the stage for other translations from refinement systems to type theory.

1 Introduction

Refinement types checkers, such as Liquid Haskell [39], Stainless [22], F* [37], and Flux [25], rely on SMT solvers to reason about program properties. They work on top of existing, industrial programming languages, allowing programmers to work in familiar environments while enjoying a high degree of automation in program verification. However, practical refinement types are disconnected from their theoretical foundations. Even though program behaviors and SMT logics are very well understood separately, the connection between them has not been made explicit.

In refinement types, SMT validity checking is used to decide subtyping. To do so, a connection between program behaviors and SMT logics needs to be made. As an example, consider an expression e with a type that ensures it is an integer greater than 5, i.e., $e : \{x:\text{Int} \mid 5 < x\}$. Since $5 < x$ implies $0 < x$, for all integers x , e can also be used at type $\{x:\text{Int} \mid 0 < x\}$. This reasoning is captured by the following two rules:

$$\frac{\Gamma \vdash e : \tau_l \quad \Gamma \vdash \tau_l <: \tau_r}{\Gamma \vdash e : \tau_r} \text{ T-SUB} \qquad \frac{\text{SMT-Valid } (\llbracket \Gamma \rrbracket \Rightarrow \llbracket r_l \rrbracket \Rightarrow \llbracket r_r \rrbracket)}{\Gamma \vdash \{x:B \mid r_l\} <: \{x:B \mid r_r\}} \text{ S-BASE}$$

The left rule, T-SUB, allows for *implicit subtyping*: an expression e of type τ_l can be given the type τ_r , if τ_l is a subtype of τ_r . To decide subtyping, refinement type systems reduce it to SMT validity checking using a set of subtyping rules, that ultimately boil down to the rule S-BASE on the right. This rule states that a refinement type $\{x : B \mid r_l\}$ is a subtype of $\{x : B \mid r_r\}$ if the logical implication $r_l \Rightarrow r_r$ is valid in the SMT logic and under the assumptions in the type environment Γ .

The critical part is: program elements are translated to the SMT logic via the embedding function $\llbracket \cdot \rrbracket$. In cases where both program elements and logics are the same (e.g., integers with arithmetic operations), the embedding function is trivial. For example, when relating $\{x:\text{Int} \mid 5 < x\}$ and $\{x:\text{Int} \mid 0 < x\}$. However, in general, defining the embedding function is intricate, and the details of its definition have not been formally accounted for.

Formalization of refinement type systems have provided partial descriptions of only the critical parts of the embedding function [22, 25, 37, 39]. Accepting that it is impractical to formally specify the actual embeddings of real refinement type checker implementations, we take a different approach to establish solid foundations for SMT-based verification: *translation to foundational type systems*.

We present Lava, a translation from the refinement type system Liquid Haskell to Rocq. Lava connects the two worlds of programs and SMT logics by modeling both in the formal system of Rocq and explicitly embedding program elements into logical ones. Thus, our translation offers both *a)* deeper insights into the foundations of refinement type checking and *b)* an automated certificate checker for Liquid Haskell verified programs. Our automated checker is able to automatically verify sizable examples, but crucially allows the user to manually intervene whenever automation fails.

1.1 Key Challenge of the Translation: Coexistence of Refined and Unrefined Semantics

The main difficulty in our translation is that Liquid Haskell has multiple semantic levels that coexist in programs and serve different purposes. Consider for example the following snippet in which a binder x , which is specified via a refinement type to be greater than 5, is used to divide 42:

```
let x :: {x:Int | 5 < x} = 8 in div 42 x -- div :: Int → {v:Int | 0 < v} → Int
```

Refinement types restrict the domain of a type, here Int , with a logical predicate. In div and below, we write Int for the trivially refined $\{v:\text{Int} \mid \text{True}\}$.

In this snippet, the binder x has three different semantics. When the snippet is seen as a Haskell program, the binder x is a *a) program integer* that can be evaluated to 8. But Liquid Haskell also attaches to x the information that it is greater than 5. Thus, in type theoretical terms, x is a *b) dependent pair* storing the evidence that the integer attached to x is greater than 5. Finally, at the call site of the division, Liquid Haskell will ensure that the arguments do not violate the specification of the function. In that case, since the second argument of division is specified to be greater than 0, it will create the verification condition $\forall x, 5 < x \Rightarrow 0 < x$ and check it using an SMT solver. When passed to the SMT solver, the binder x is a *c) logical integer* that admits arithmetic operations.

Two parallel worlds coexist:

- The *refined world* of the Liquid Haskell type checker, where x is the *b)* dependent pair, and
- The *unrefined world* where x is a *a)* program integer or *c)* logical integer.

These two semantics are crucial for how refinement types work in Liquid Haskell. The Liquid Haskell type checker uses *refined definitions* in the program to generate the verification conditions (VCs), while the SMT solver uses *unrefined definitions* in the refinements to solve the VCs.

The key challenge of the translation is to account for these two intertwined semantics, while establishing a correspondence between the refined and unrefined translations of the same object.

1.2 Refined and Unrefined Semantics in Functions

The coexistence of refined and unrefined semantics becomes even more delicate once we move from simple values to functions. Functions in Liquid Haskell simultaneously inhabit both worlds: they are defined as Haskell programs with refinement type specifications, but are also used into the logical layer to specify refinements. This dual nature of functions leads to unexpected behaviors that must be captured by the translation.

As an example of an unexpected behavior, consider a function that gets an element from a list.

```
{-@ get :: l:IList → i:{Int | 0 ≤ i ∧ i < llen l} → Int @-}
get :: IList → Int → Int
get (Cons x l) i = if i == 0 then x else get xs (i-1)
```

The function `get xs i` returns the i -th element of the monomorphic list xs . The Haskell definition is partially defined, so a refinement type specification ensures that the index is always in-bounds. The full definitions and detailed translation to Rocq is given in § 2.5. Here we note that variant of this `get` function is the main motivation for refinement types: they ensure the lack of runtime errors without the need to explicitly handle out-of-bounds cases in the function definition.

When `get` is used in Haskell programs, it has the *refined semantics*: each call `get xs i` generates a VC ensuring that the index `i` is in-bounds. However, when `get` is used in refinements, it is treated as an SMT logical function, thus, cannot be partial.

As a concrete, surprising example, we prove that getting the 4th element of `Cons x l` is the same as getting the 3rd element of `l`, for any list `l`!

```
{-@ surprise :: x:Int → l:IList → {u: () | get (Cons x l) 4 = get l 3 } @-}
```

Liquid Haskell accepts this definition, even though the specification of `get` is not satisfied for arbitrary lists `l` which might be shorter than length 4. The specification of `surprise` refines the result unit type to express the equality `get (Cons x l) 4 = get l 3`. Here, inside the refinements, `get` is a total, logical function that *models* the behavior of the program function `get`.

This dual nature of functions in Liquid Haskell is challenging to capture in a Rocq translation—Rocq quite rightly does not let you apply functions outside of their domain! Lava has to not only define both the refined and unrefined versions of `get`, but also relate them properly.

1.3 The Design of Lava

We designed Lava, a translation from well-typed Liquid Haskell programs to Rocq that respects the semantics of Liquid Haskell by carefully accounting for both the coexistence of refined and unrefined semantics in Liquid Haskell programs as well as implicit subtyping.

Modeling Refined and Unrefined Semantics. To translate functions, like `get`, Lava generates two objects. One object models the *refined semantics* of the function, as a dependently typed Rocq function that includes the necessary proofs to ensure that the function respects its refinement type specification. The other object models the *unrefined semantics* of the function, as a Rocq relation that captures the graph of the function. Finally, Lava also generates and proves lemmas that relate the two semantics, ensuring that the refined function behaves according to its unrefined graph relation. To support higher-order functions, a refined higher-order function needs to take as input not only the refined variant of its input, but also the unrefined variant, a proof that the unrefined variant is a functional relation, and a proof connecting the two. We call this “function plus graph plus two proofs” a *Pack*; packing these four components together simplifies function subtyping.

Modeling Implicit Subtyping with Explicit Coercions. When translating Liquid Haskell programs, Lava systematically uses casts to model implicit subtyping, *i.e.*, the typing rule T-SUB which lies at the core of many refinement type systems. Lava translates program definitions based on the type checking derivation tree and translates implicit subtyping to explicit type coercions. Those type coercions generate proof obligations. Along with the translation itself, we also provide a custom automation tactic that we call `solver` designed to discharge these obligations. The `solver` tactic is able to automatically discharge the proof obligations generated by our case studies (§ 6), closely mirroring the level of automation provided by Liquid Haskell’s SMT backend.

Implementation. We implemented Lava, a translation from programs verified in Liquid Haskell to Rocq. The implementation consists of three main components. First, the Haskell code implementing the “*skeleton*” translation of terms to Rocq. This translation is the main subject of the paper and it is presented over a core calculus for refinement types λ_r that is a subset of Liquid Haskell. In reality Lava translates Liquid Haskell terms to their refined and unrefined Rocq counterparts and translates implicit subtyping to explicit coercions. Second, a Rocq library with the definitions necessary to support our translation. Third, a collection of Rocq (Ltac) automation tactics that prove obligations due to refinements (in particular the `solver` tactic) or prove structural lemmas, notably those relating unrefined and refined terms.

Contributions. We contribute the design, metatheory, and implementation of Lava:

- (1) **A foundational semantics for monomorphic Liquid Haskell** (§ 3): The translation of verified Liquid Haskell programs to Rocq formalizes a semantics of Liquid Haskell and crystallizes the need for the two intertwined semantics. Our design supports monomorphic programs, with user defined data types and higher-order functions. The intricacy of the semantics lies in making all fine-grained details work together as a whole and required significant fine-tuning.
- (2) **A correctness theorem for the translation** (§ 4): We prove that the translation preserves the semantics of declared functions and types (theorems 4.1 and 4.2) and that inconsistencies are preserved (theorem 4.3), so Lava can be used as a certificate checker for Liquid Haskell. The meta-theory sheds light on how to design such a translation for similar sources and targets.
- (3) **An implementation and evaluation of Lava** (§ 5 and 6): Our tool consists of around 5K SLOC of Haskell code implementing the skeleton translation, and around 6.5K SLOC for the automation tactics and Rocq library. The implementation of the tool and automation tactics is a significant engineering effort. We evaluate the translation on eight representative Liquid Haskell programs.

While our work focuses on Liquid Haskell— and, more marginally, on Rocq— we believe it reveals fundamental aspects of the semantics of refinement type systems. It can be used as a basis for the translation of other refinement type frameworks to proof assistants.

2 Overview of the Translation

We begin with an informal overview of Lava, presenting the key features of Liquid Haskell and how they translate to Rocq. Our running example is a “safe” indexing function on lists—one that can only be applied to valid indices of a list (§ 2.5). For exposition purposes, our indexing function arbitrarily requires lists containing integers larger than 5; we prove a theorem relating indexing and list concatenation (§ 2.7). Along the way, we translate builtin types and refinements (§ 2.1), inductive types (§ 2.2), and recursive (§ 2.3 and 2.4) and higher-order functions (§ 2.6). This section should be accessible to readers with a superficial knowledge of either Liquid Haskell or Rocq.

2.1 Translation of Refinement Types

Liquid Haskell is a refinement type system for subset types built atop Haskell and it adopts Haskell’s underlying norms. Refinement types (*aka subset refinement types*) $\{x:B \mid \phi(x)\}$ denote a set of elements x of type B such that $\phi(x)$ holds. Rocq also supports refinement types, using the same notation to denote a (dependent) pair of an element of $e:B$ and a proof of $\phi(e)$.

Our list indexing example depends on integers. Liquid Haskell comes with a builtin integer type, `Int`, with over- and underflow left as undefined behavior. We translate these underspecified machine `Ints` to Rocq’s unbounded integer type `Z`. We translate the Liquid Haskell type of integers greater than five— $\{x:\text{Int} \mid 5 < x\}$ —to Rocq (definition 3.6) as:

Definition `Z_gt_5` := $\{x:Z \mid <_{\text{rel}} 5 \ x \ \text{true}\}$.

Refinement types translate to Rocq’s subset types, so refinements translate to Rocq’s propositions. Here, $5 < x$ turns into a proposition about `<_rel`, the three-way relation modeling `<` on integers: $(5, x, \text{true}) \in <_{\text{rel}}$ if and only if $5 < x$. The translation of refinements to propositions (definition 3.4) is subtle, since it requires the translation of function calls, such as $5 < x$ in our example.

Given $x : Z$ and $p_x : <_{\text{rel}} 5 \ x \ \text{true}$, we build a value of this type in Rocq as `exist _ x p_x`, where `exist` is the constructor for subset types. The hole `_` is filled automatically by Rocq in our examples, with the type of the proof: $\forall(x:Z), <_{\text{rel}} 5 \ x \ \text{true}$.

2.2 Translation of Refined Inductive Types

Having defined integers greater than five (Z_{gt_5}), let us move on to our example inductive datatype: lists of such integers. Liquid Haskell programs can refine Haskell's inductive datatypes with predicates on both the argument and return types of the constructors. Such definitions come in two parts: a *refined definition* (in Liquid Haskell's special comment form $\{-@ \dots @-\}$) and an *unrefined definition* in plain Haskell:

```
{-@ data IList where
  Nil :: IList
  Cons :: x:{x:Int | 5 < x} → IList → IList @-}
data IList where
  Nil :: IList
  Cons :: Int → IList → IList
```

For the supported types, mapping unrefined definitions to Rocq amounts to a change in syntax:

```
Inductive IList_u := Nil_u : IList_u | Cons_u : Z → IList_u → IList_u.
```

One might be tempted to define the refined type inductively, as well. For our example of lists of numbers greater than 5, this is possible:

```
(* XXX not how our translation works! *)
Inductive IList_XXX := Nil : IList_XXX | Cons : ∀ (x : Z_gt_5) → IList_XXX → IList_XXX.
```

But the approach does not work in general: Liquid Haskell allows refinements of one constructor argument to mention other constructors. That is, Liquid Haskell would allow us to define a constructor like the following:

```
{-@ ... ConsNonEmpty :: {x:Int | 5 < x} → {xs : IList | xs <> Nil} → IList @-}
```

Rocq's type theory forbids such a refinement: we cannot use a constructor *while defining the type*. To keep our datatype translation uniform, we always translate a refinement of an inductive datatype to a *recursive refinement predicate* [3, 31] (§ 3.3.1):

```
Fixpoint IList_wf (xs : IList_u) : Prop :=
  match xs with
  | Nil_u ⇒ True                                (* refinement for the conclusion (trivial) *)
  | Cons_u x xs' ⇒ <_rel 5 x true                (* refinement on x (5 < x) *)
                  ∧ True                        (* refinement for xs' (trivial) *)
                  ∧ IList_wf xs'                (* recursion on xs' *)
                  ∧ True                        (* refinement for the conclusion (trivial) *)
  end.
```

In Rocq, the keyword for a recursive function is `Fixpoint` and the keyword for case analysis is `match`. Our predicate accounts for many refinements: each constructor argument could be refined; each recursive reference to the type is refined *by the predicate we are defining* and possibly another one (as in `ConsNonEmpty`, above); and the return type of the constructor could be refined. We write the trivial refinement `True` if there is no explicit refinement.

Refinement predicate in hand, we can finally output the refined definition of the type `IList`, along with refined versions of its constructors:

```
Notation IList := {xs : IList_u | IList_wf xs}.
Definition Nil : IList := exist_ Nil_u ltac:(solver).
Definition Cons (x : Z_gt_5) (xs : IList) : IList := exist_ (Cons_u [x] [xs]) ltac:(solver).
```

The refined `IList` is just a subset type of `IList_u` with the predicate `IList_wf`. `Nil` is just an `IList`, while `Cons` takes two refined arguments and passes them to the unrefined constructor `Cons_u`. It uses the projection operator `⌊ · ⌋` (here a notation for Rocq’s `proj1_sig`) to extract the underlying value of the dependent pairs `x` and `xs`. For the proof of the refinements, we use an inline tactic `ltac:(solver)`—we expect to automatically resolve the refinement proof witness. Liquid Haskell uses an SMT solver for automation; in Rocq, our translation invokes our automation tactic `solver`.

For every datatype, we generate a typeclass instance of an equality `==?` (along with a `Prop`-valued relation, `==`) for structural equality, which is reflexive and implies Rocq’s Leibniz equality.

2.3 Translation of Functions with Dependent Pattern Matching

Functions in Liquid Haskell typically come in two parts, with an explicit refinement type as specification (in comments) atop a conventional Haskell definition:

```
{-@ llen :: IList → {v: Int | 0 ≤ v} @-}
llen :: IList → Int
llen Nil = 0
llen (Cons _ xs') = llen xs' + 1
```

The `IList` argument is treated as trivially refined with `True`: non-arrow types in Liquid Haskell are always refined. We translate it to Rocq simply as the subset type `IList`.

The function `llen` refines its codomain to be non-negative ($0 \leq v$), which gets translated to $\leq_{\text{rel}} 0 \vee \text{true}$, the corresponding inductive relation. We could translate `llen` to a Rocq `Fixpoint` with a refined return type, but *unrefined* inputs and explicit proof parameters:

```
(* XXX not our actual translation *)
Fixpoint llen_rec_XXX (xs_u : IList_u) {struct xs_u} : IList_wf xs_u → {v : Z | ≤_rel 0 v true}.
  refine (match xs_u with
    | Nil_u      ⇒ λxs_p, exist _ _
    | Cons_u _ xs_u' ⇒ λxs_p, let (v, p) := llen_rec_XXX xs_u' _ in exist _ (v + 1) _
  end).
all: solver. Defined.
Definition llen_XXX (xs : IList) : {v : Z | ≤_rel 0 v true} :=
  let (xs_u, xs_p) := xs in llen_rec_XXX xs_u xs_p.
```

The fixpoint definition is given as a *proof script* terminated by `Defined`. This allows us to use Rocq’s `refine` tactic to give a partial definition with some *proof holes* (indicated by `_`); the solver tactic neatly dispatches those holes. The proof holes only stand for proof obligations on refinements, never for actual values of the program.

Readers unfamiliar with Rocq may be surprised by our split definition. In Rocq, recursive definitions need to be directly defined on inductive types with syntactically clear termination. We break up the dependent pairs on the input into their constituent parts: an unrefined value `xs_u : IList_u`, on which we perform the recursion, and a proof `xs_p : IList_wf xs_u`. We then define `llen_XXX` on top of `llen_rec_XXX`.

Still, Rocq’s `match` construct does not propagate case analysis information to the context. Thus, each branch must return a function (starting with `λxs_p`). For more complex examples, maintaining all of this in the translation becomes burdensome—dependently typed programming in Rocq is hard! We decide to translate the structure of definitions with tactics and restrict the use of `refine` to a more elementary section of the grammar (§ 3.5.1):

```
Definition llen (xs : IList) : {v : Z | ≤_rel 0 v true}.
  destruct xs as [xs_u xs_p]. induction xs_u as [(Nil*) | (*Cons*) x xs_u' rec].
```

```

- refine (exist (V(v:Z), ≤rel 0 v true) 0 _).
- refine (subsumptionCast (V(v:Z), ≤rel 0 v true) (
  (subsumptionCast (V(_:Z), True) (rec _)) +Z (exist (V(_:Z), True) 1 _))); solver.
Defined.

```

The proof code generates morally the same term in the kernel language of Rocq as the manual definition above, up to proof irrelevance. Once again, we break up the input xs into the unrefined value xs_u and a refinement witness xs_p for it, using the `destruct` tactic. Then we proceed by `induction` on the unrefined value xs_u . Using `induction` in a “definition by tactics” gives us dependent pattern matching, with propagation of the case analysis information. As well as an inductive hypothesis `rec`, already instantiated for xs_u' , that we can use to do the same recursion as in a conventional `Fixpoint` definition.

The branches are translated directly as terms, as they are simple expressions made only of integers, addition and the recursive call. We use operators on dependent pairs (here $+Z$) to be faithful to Liquid Haskell semantics. Two subsumption casts in the second branch correspond to two subtyping obligations in Liquid Haskell, and their proofs are inferred by solver.

We could have used frameworks like Program [32] or Equations [33] to define `llen`. Instead, we choose to rely only on basic Rocq tactics, which are flexible, reliable, and let us easily slot in our own solver as needed—a tradeoff that works for our implementation (§ 7).

2.4 Translation of Functions as Unrefined Graph Relations

Liquid Haskell permits the user to *reflect* functions into the SMT logic [40], enabling reasoning about their definitions. For reflected functions, Lava generates two definitions in Rocq: a *refined* definition (as presented before) and an *unrefined* graph relation that approximates the function behavior. We choose relations as a simple representation of partial functions in Rocq. We translate a possibly recursive function of type, e.g., $A \rightarrow B$, to an (inductive) relation $A \rightarrow B \rightarrow \text{Prop}$; in general, an n -ary function turns into an $n+1$ -ary relation (§ 3.3.2). In our example, `llen` is reflected, using the annotation `{-@ reflect llen @-}`; thus we further generate the following graph relation:

```

Inductive llen_rel : llist_u → Z → Prop :=
| llen_rel_Nil : llen_rel Nil_u 0
| llen_graph_Cons x xs' : ∀ v, llen_rel xs' v → llen_rel (Cons_u x xs') (v + 1).

```

Crucially, we automatically prove (using Lava’s custom tactics) that the graph denotes a function (*functionhood*) and that functions and their graphs coincide:

```

Lemma llen_funct x v v' : llen_rel x v → llen_rel x v' → v = v'. Proof. ... Qed.
Lemma llen_llen_rel xs v : [llen xs] = v ↔ llen_rel [xs] v. Proof. ... Qed.

```

When we translate a Liquid Haskell refinement type to Rocq, we *always* use these unrefined graph relations: we must do so to be faithful to Liquid Haskell’s semantics, since *automation in the SMT solver uses unrefined definitions in the refinements*. For uniformity, we also translate builtin functions to relations. For example, $(<) : \text{Int} \rightarrow \text{Int} \rightarrow \text{Bool}$ corresponds to a relation $<_{\text{rel}}$. This is how the refinement $5 < x$ from § 2.1 translates to the Rocq proposition $<_{\text{rel}} 5\ x\ \text{true}$.

In general, we might need to introduce fresh variables to stand for the result of a computation in the relation. This is the case in the definition of `llen_rel` itself, where the result of the recursive call `llen xs'` is captured by the fresh variable v in the hypothesis `llen_rel xs' v`.

For every top-level function, we create a typeclass instance (in Rocq, a record value [34]) containing the refined definition, the unrefined graph relation, the lemma relating the two and the

functionhood property. The typeclass is called a *Pack*, and is defined parametrically over an arrow type¹. We need them to deal with higher-order features, as we detail in § 2.6. For `llen`, we have:

```
Instance llen_pack : Pack (IList → {v:Z | ≤rel 0 v true}) :=
  { def := llen; rel := llen_rel; f_f_rel := llen__len_rel; funct := llen_funct }.
```

2.5 Translation of the Safe Indexing Function

We have arrived at the heart of our example: the safe-indexing function `get`:

```
{-@ get :: xs:IList → i:{i:Int | 0 ≤ i ∧ i < llen xs} → {v:Int | 5 < v} @-}
get :: IList → Int → Int {-@ reflect get @-}
get (Cons x xs') i = if i == 0 then x else get xs' (i-1)
```

Note how `get` has just one defining equation—there is no need for a `xs = Nil` case, ruled out by the safety constraint $0 \leq i \wedge i < \text{llen } xs$. To translate `get` to Rocq, we must not only translate a recursive definition with a non-trivial refinement type, we must do so *while proving that the function is total*. Our implementation generates the following code:

```
Definition get (xs : IList)
  (i : {i : Z | ≤rel 0 i true ∧ ∀ llen_l, llen_rel [xs] llen_l → <rel i llen_l true})
  : {v : Z | <rel 5 v true}.
  destruct xs as [xs_u xs_p]. destruct i as [i_u i_p].
  generalize dependent i_u; induction xs_u as [x xs_u' rec]; intros.
  - (* xs_u = Nil_u *) exfalso. solver.
  - (* xs_u = Cons_u x xs' *) destruct (i_u ==? 0) eqn:?.
    + refine (exist (∀ (v:Z), <rel 5 v true) x _); (* 5 < x *) solver.
    + refine (rec _ (i_u - 1) _).
      (* IList_wf xs' *) solver.
      (* 0 ≤ i_u - 1 ∧ ∀ llen_l, llen_rel xs' llen_l → i_u - 1 < llen_l *) solver.
Defined.
```

How does this definition work? First, the `destructs` decompose `xs` and `i` into unrefined values (`xs_u`, `i_u`) and refinement proofs (`xs_p`, `i_p`). We then re-generalize (`generalize dependent`) the underlying index `i_u` of the refinement `i_p`. (A paper proof would simply say, “leaving `i_u` general”.) At this point, we have defined the “wrapper” of `get`; we now use `induction` to define the fixpoint on the unrefined list `xs_u` (`intros` introduces forall quantifiers).

The `induction` tactic generates the core pattern match in our function: `xs_u` is either `Nil_u` or `Cons_u`. If `xs_u = Nil_u`, we have a contradiction: $0 \leq i_u \wedge i_u < \text{llen Nil}$, but $\text{llen Nil} = 0$. We indicate unreachability of the branch with the `exfalso` tactic: if we can prove `False` (*ex falso...*), we can prove anything (*...quodlibet*). The `solver` tactic identifies the contradictory assumptions and applies the principle of explosion appropriately—just as SMT would for Liquid Haskell.

On the other hand, if `xs_u = Cons_u x xs_u'`, we generate tactics corresponding to the Liquid Haskell term—here, a conditional. We use `destruct` to go by cases on whether `i_u` is 0, translating (`==`) in Liquid Haskell to `IList`’s (`==?`) decision procedure in Rocq (§ 2.2). When `i_u` is 0, we return `x`—using the `refine` tactic to provide a partial term to be filled in by `solver`. Here we need only show that `x` will satisfy the postcondition $5 < x$ of `get`, which `solver` handles easily enough. But if $0 < i_u$, we continue with a recursive call, given by the induction hypothesis `rec` already instantiated for `xs_u'`. We have left holes for the refinement witnesses of `xs_u'` and `i_u` (writing their types in comments); `solver` neatly fills both.

¹In practice, the parameters of *Pack* are encoded as a dependent list of arguments to capture the refinement types.

We also need to define `get`'s graph relation. We translate the graph following Haskell control structure directly, resulting in a Rocq definition that is easier to read but hard to compute with:

```
Inductive get_rel : IList_u → Z → Z → Prop :=
| get_Cons_true i x l': i == 0 → get_rel (Cons_u x xs') i x
| get_Cons_false i x l': i /= 0 →
  ∀ (j: Z), subZ_rel i 1 j → ∀ (n: Z), get_rel xs' j n → get_rel (Cons_u x xs') i n.
```

Finally, we need to connect the refined and unrefined definitions, prove functionhood of `get_rel`, and instantiate a `Pack` for `get` (where `get_tp` stands for the translated type of `get`):

```
Lemma get__get_rel (xs : IList)
  (i : {i : Z | ≤_rel 0 i true ∧ ∀ llen_l, llen_rel [xs] llen_l → <_rel i llen_l true}) v:
  [get xs i] = v ↔ llen_rel [xs] [i] v. Proof. ... Qed.
Lemma get_func xs i v v' : get_rel xs i v → get_rel xs i v' → v = v'. Proof. ... Qed.
Instance get_pack : Pack (get_tp) :=
{ def := get; rel := get_rel; f_f_rel := get__get_rel; funcnt := get_func }.
```

2.6 Translation of Higher-Order Functions

`Pack` instances, like the ones of `llen` and `get`, are necessary when functions are used as first-class values. As an example of this necessity, consider a refined `apply` function.

```
{-@ apply :: f:(IList → {v:Int | 0 ≤ v}) → xs:IList → {v:Int | v = f xs} @-}
apply :: (IList → Int) → IList → Int {-@ reflect apply @-}
apply f xs = f xs
```

The functional argument `f` appears unrefined in the output refinement, but refined in the body of `apply`. Translating the arrow to a `Pack` allows us to retrieve both versions of the function `f`.

```
Definition apply (f: Pack (IList → {v:Z | ≤_rel 0 v true})) (xs: IList)
  : {v: Z | ∀ res, f.(rel) fst res → v = res}.
destruct xs as [xs_u xs_p]. refine (subsumptionCast _ _ (f.(def) (exist _ x _))); solver. Defined.
```

In the refinements, we use the graph relation `f.(rel)` from the `Pack`, while in the body we use the refined function `f.(def)`.² The function `apply` can be applied to `llen_pack`, that is an instance of the required typeclass. Then, the projection `f.(rel)` will be instantiated to `llen_rel`, while `f.(def)` will be instantiated to `llen`.

In the unrefined world, when building the relation `apply`, we also use a `Pack` for the higher-order arguments, this time an *unrefined* `Pack`. Unrefined `Packs` only contain the two unrefined elements of the `Packs`: the relation (used below with `f.(rel_u)`) and the functionhood property.

```
Inductive apply_rel : uPack (IList_u → Z) → IList_u → Z → Prop :=
| apply_f_x f x v: ∀ res, f.(rel_u) x res → applyToFirst_rel f x v.
```

To instantiate `apply_rel` with `llen`, we define a projection from refined to unrefined `Packs` (here from `Pack (IList → {v:Z | ≤_rel 0 v true})` to `uPack (IList_u → Z)`). This projection simply forgets two of the fields to keep only the graph relation and functionhood property.

²We simplify here. Since `Packs` are defined over dependent lists of arguments, instead of sequences of arguments, the implementation actually uses wrapper functions (e.g., `getPackRel`) instead of the direct projections (e.g., `f.(rel)`).

2.7 Translation of Theorems Definitions and Invocations

Liquid Haskell theorems are simply functions returning a refinement of unit—there is no `Prop` type in Liquid Haskell. In the absence of side-effects, all functions returning the unit `()` are considered theorems. Here, we prove a simple lemma relating `get` and `Cons`:

```
{-@ thm_get_cons :: xs:IList → x:{v:Int | 5 < v}
    → i:{i:Int | 0 ≤ i && i < llen xs}
    → {-:() | get xs i == get (Cons x xs) (i+1)} @-}
thm_get_cons :: IList → Int → Int → ()
thm_get_cons _ x _ = trivial
```

We opt for the simplest possible design here: as theorems in Liquid Haskell are just programs, we translate them like programs. But because the proof of the program itself is irrelevant, theorems are never reflected. They are also never executed or called in a refinement. Therefore, we do not bother generating the graph relation for theorems, not even the generic one for unreflected functions. We translate `thm_get_cons` as follows:

```
Definition thm_get_cons (xs : IList) (x : {x: Z | <_rel 5 x true})
  (i : {i: Z | ≤_rel 0 i true ∧ ∀ llen_l, llen_rel [xs] llen_l → <_rel i llen_l true}) :
  {-: Unit | ∀ v, get_rel [xs] [i] v → ∀ j, addZ_rel [i] 1 j
    → ∀ w, get_rel (Cons_u [x] [xs]) j w → v == w}.
intros. destruct xs as [xs_u xs_p]. destruct x as [x_u x_p]. destruct i as [i_u i_p].
refine (exist _ unit _); solver.
Qed.
```

The proof is straightforwardly generated from the Liquid Haskell syntax: introduce variables (`intros`), break refinements into values and witnesses (`destruct`), return a unit (`refine`), and let the solver resolve the proof (`solver`).

Liquid Haskell allows inductively proving a theorem by pattern matching on the arguments. Such a proof is translated to a dependent pattern matching in Rocq as we saw for recursive functions. Liquid Haskell also offers a few combinators for proof writing, of which we translate the most important ones. For example, we can give an expression `e2` as a hint to a proof `e1` with `e1 ? e2`, which is translated in Rocq as an assertion to the proof context.

3 The Translation

3.1 Elements of the Translation and Internal Representation

Lava translates Liquid Haskell to Rocq in three steps. The first step elaborates Haskell’s internal representation Core [36] and Liquid Haskell specifications into our calculus λ_r (presented below).

The second step annotates the terms of λ_r to prepare the translation. Liquid Haskell uses implicit subtyping [28] and binders can appear as refined or unrefined according to the context. We give a refinement type system for λ_r that converts subtyping constraints to type casts in λ_r -terms (§ 3.2).

The third step is the translation of annotated terms into Rocq (as detailed in § 3.3 to 3.5).

3.1.1 Syntax of λ_r . Figure 1 presents the syntax of λ_r , a core λ -calculus with refinement types, user-defined first-order inductive data types and functions. The core grammar of λ_r is in black, while the extended grammar includes the terms in gray. This work does not cover the following features of Liquid Haskell: polymorphism, partial application and lambdas in refinements or reflected functions, data constructors with higher-order parameters, and non-structurally recursive functions.

The calculus λ_r uses the metavariables TC and C for type and data constructors, while the term variables are drawn from the set of all variables $\mathcal{V} \supseteq \{x, y, z, f, g, v\}$. The name v is reserved for

Base Types	$B ::= () \mid \mathbf{Bool} \mid \mathbf{Int} \mid TC$
Refinement Types	$R ::= \{x:B \mid r\} \mid x:R \rightarrow R$
Declarations	$D ::= \mathbf{data} \ TC := (C :: R)^+ \mid (\mathbf{reflect})^? \ \mathbf{def} \ f :: R := e$
Expressions	$e ::= r \mid \mathbf{let} \ x :: R := e \ \mathbf{in} \ e \mid \mathbf{if} \ r \ \mathbf{then} \ e \ \mathbf{else} \ e$ $\mid \mathbf{case} \ r \ \mathbf{of} \ (C \ \bar{x} \mapsto e)^+ \mid \mathbf{unreachable}$
Refinements	$r ::= x_{/(n,c)} \mid \ell \mid C \mid rr \mid \neg r \mid r_1 = r_2 \mid r_1 \odot r_2 \quad c \in \{G, L, Y\}$ $\mid r ? (r \text{ proves } r) \mid \mathbf{inj}(r, R) \mid \mathbf{sub}(r, R, R) \mid \mathbf{proj}(r)$
Literals	$\ell ::= () \mid \mathbf{True} \mid \mathbf{False} \mid 0, -1, 1 \dots$
Binary Operators	$\odot ::= + \mid - \mid \times \mid / \mid < \mid \leq \mid > \mid \geq \mid \Rightarrow \mid \wedge \mid \vee$

Fig. 1. Syntax of λ_r ; its monomorphic base types B are built into more general refined types R . We write $\overline{(\cdot)}$ for a sequence of zero or more elements, $(\cdot)^+$ for a non-empty sequence and $(\cdot)^?$ for an optional annotation. In gray, the extended grammar after typing.

binders in refinement types. In the extended syntax, variables are annotated with their arity (n) and their scope: whether they are top-level (G), local (L), or the name of the function being defined (Y).

We define λ_r 's *types* in two layers: base and refinement types. A *base type* (B) is either builtin (unit, boolean, or integer) or a user-defined type constructor. A *refinement type* (R) is either a base type refined by a proposition or a function between two refinement types. In function types, the codomain's refinement can refer to the domain. We abbreviate the trivial refinement $\{x:B \mid \mathbf{True}\}$ to $\{B\}$; we abbreviate $x_1:R_1 \rightarrow \dots \rightarrow x_n:R_n \rightarrow R$ to $(x_i:R_i)_{i \leq n} \rightarrow R$, where n is maximal.

A λ_r program is a sequence of *declarations* (D), where each declaration defines either a type constructor or a function. Just like in Haskell, every function can be recursive; just like in Liquid Haskell, every function can be *reflected* into the logic. Top-level function definitions contain implicit λ -binders and are η -long: $(\mathbf{reflect}) \ \mathbf{def} \ f :: x_1:R_1 \rightarrow \dots \rightarrow x_n:R_n \rightarrow \{x:B \mid r\} := e$ binds the formal parameters x_1 through x_n of f . Type constructor definitions must be first-order: every constructor's arguments must be refinements of base types, not arrow types.

The body of a definition is a λ_r *expression* (e). We use a form of generalized ANF [29], where expressions are broken into two levels: refinements r and structural expressions e . The upper level contains let-bindings, conditionals and pattern matchings. Let-bindings are annotated with a refinement type: like in top-level declarations, if the type is an arrow, it binds the formal parameters of e . In other words, all λ -abstractions are implicit and let-bound. Pattern matchings can be partial, but all missing branches are signaled as **unreachable**.

Refinements r are both expressions and type refinements. We distinguish boolean equality from binary operators. *Binary operators* ($\odot$) are primitives for arithmetic, equality, and boolean logic. *Literals* (ℓ) are unit, booleans, and integer values. *Proof hints* ($r ? r_h$) provide hints (to the SMT solver) to prove theorems. The extended grammar has three type casts: *injections* from base to refined types, *subsumptions* over refinement types, and *projections* from refined to base types.

3.1.2 Structure of the translation from λ_r to Rocq. The translation of λ_r is performed by the seven functions summarized in table 1. Liquid Haskell's semantics forces us to work in two "worlds" (§ 2): an *unrefined* world with type refinements and casts stripped away; and a *refined* world, where refinements translate to Rocq propositions and with type casts.

The translation follows the structure of a Liquid Haskell file and translates each datatype or function declaration independently using the function `DtoR` (§ 3.3) that outputs both a refined and an unrefined version of the declaration. For types, refinements, and expressions, there are two dual sets of translations: to the unrefined world (§ 3.4) and to the refined world (§ 3.5). In

Table 1. The core functions of the Lava translation from λ_r to Rocq.

Target World	Element of λ_r	Function	Definition
Refined and Unrefined	Declaration	DtoR	Figures 2 and 3
Unrefined	Type	TtoU	Definition 3.1
	Refinement	RtoU	Definition 3.2
	Refinement (as Prop)	RtoP	Definition 3.4
Refined	Type	TtoR	Definition 3.6
	Refinement	RtoR	Definition 3.8
	Expression (as tactics)	EtoTac	Definition 3.7

the unrefined world, refinements are translated to propositions (RtoP) when inside a type and to arbitrary terms (RtoU) when part of an expression. In the refined world, expressions are translated as tactics (EtoTac). Some of the functions have an additional parameter, described in § 3.5.

3.2 Elaboration of Explicit Type Casts

Before translating λ_r to Rocq, Lava performs an elaboration phase that turns all implicit subtyping into explicit type casts, through well-formedness and typing derivation rules that mirror the rules of Liquid Haskell. The type system itself poses no surprise, the interesting feature is the elaboration; we present the rules with a non-trivial elaboration. The rest is in appendix A.

As a running example, we assume the declarations from the overview (§ 2) and consider the elaboration of **def** $f :: R_f := \text{get } (\text{Cons } n \text{ Nil}) \ 0$, with $R_f \doteq n : \{n : \mathbf{Int} \mid 8 < n\} \rightarrow \{v : \mathbf{Int} \mid v = n\}$.

Elaboration of refinement types. Since types appear in the elaborated expressions, their refinement must also be annotated for the sake of translation. In our example, R_f elaborates to $R_f^e \doteq n : \{n : \mathbf{Int} \mid 8 < n_{/(0,L)}\} \rightarrow \{v : \mathbf{Int} \mid v_{/(0,L)} = \mathbf{proj}(n_{/(0,L)})\}$. It has two kinds of annotations: a projection and pairs arity/locality on the variables (here, all variables are local, L). The elaboration is created by checking well-formedness of the type with the following two rules:

$$\frac{\text{if } B = TC, \text{ then } B \in \text{dom}(\Gamma) \quad \Gamma, x : \{B\} \vdash_{\text{SIMPLE}} r : \mathbf{Bool} \rightsquigarrow r^e}{\Gamma \vdash \{x : B \mid r\} \rightsquigarrow \{x : B \mid r^e\}} \text{E-TRF} \quad \frac{\Gamma \vdash R_x \rightsquigarrow R_x^e \quad \Gamma, x : R_x \vdash R \rightsquigarrow R^e}{\Gamma \vdash x : R_x \rightarrow R \rightsquigarrow x : R_x^e \rightarrow R^e \{x \backslash \mathbf{proj}(x)\}} \text{E-TFUN}$$

The left rule elaborates the type refinements. It uses a simple type system that, ignoring all the refinements, simply checks that the type refinement r is boolean and annotates its variables. The annotation relies on the type environment Γ that already contains annotated variables and types.

The right rule elaborates the arrow type $x : R_x \rightarrow R$. It essentially substitutes the occurrences of the variable x in R by its projection. As type refinements live in the unrefined world, in the refinements of R , we need to extract the underlying unrefined value of x . The definition of the substitution is standard, it is capture-avoiding and assumes all bound variables are distinct.

Subsumption. We now turn to the elaboration of expressions and start with the most important and straightforward annotation: implicit subtyping echoes as a subsumption in the term.

$$\frac{\Gamma \vdash r \Rightarrow R_s \rightsquigarrow r^e \quad \Gamma \vdash R_s <: R_t}{\Gamma \vdash r \Leftarrow R_t \rightsquigarrow \mathbf{sub}(r^e, R_s, R_t)} \text{E-SUB}$$

Subtype derivations $\Gamma \vdash R_s <: R_t$ build on two rules: a contravariance rule on arrows and the base rule of § 1. Our rules, designed to mimic the Liquid Haskell rules so that they annotate the terms with the correct types and casts, do not check validity of subtyping: the base subtyping rule

has no premises and is vacuously valid. Validity is checked at two other occasions: beforehand by Liquid Haskell on the source program and afterwards by Rocq on the output.

Our type system is *bidirectional*: the types of refinements are *synthesized* ($\Rightarrow$), i.e., are an output of the judgement, while structural expressions (lets, conditionals and case) are *checked* ($\Leftarrow$) against a type. Both versions output an annotated term. All types appearing in the derivations (e.g., R_s and R_t in the subtyping rule above) outside of initial terms are annotated.

As usual in bidirectional systems, subtyping is when synthesis meets checking: the synthesized type R_s must be a subtype of the required type R_t . Since types of structural expressions are only checked, subsumption is only applied to refinements (hence r in the rule instead of e).

Injection. Two atoms of λ_r are considered unrefined: literals and local variables of nullary arity. We inject them into a refined type in the annotated term:

$$\frac{}{\Gamma \vdash \ell \Rightarrow \text{ty}(\ell) \rightsquigarrow \mathbf{inj}(\ell, \text{ty}(\ell))} \text{E-LIT} \quad \frac{(x : R) \in \Gamma \quad x \text{ is local} \quad \text{arity}(R) = 0}{\Gamma \vdash x \Rightarrow R \rightsquigarrow \mathbf{inj}(x_{/(0,L)}, R)} \text{E-VAR}$$

The left rule casts a literal into a singleton type (or *selfified* type [26]) $\text{ty}(\ell)$ (correctly annotated in an empty context). For instance, the literal 0 from our example is elaborated to $r_0^e \doteq \mathbf{inj}(0, \{v:\mathbf{Int} \mid v_{/(0,L)} = 0\})$. Literals populate the base types, which is why we consider them unrefined.

The right rule annotates variables with their locality and arity, as in type refinements. In our example, `get` is a global variable and annotated to $\text{get}_{/(2,G)}$. All top-level function or constant names are considered refined (with their type being the Liquid Haskell specification). If a local variable has an arrow type, its type does not need to be further refined.

However, local first-order variables are morally dependent pairs, that we always split in our translation. We do this to be able to match on the unrefined value. Therefore, occurrences of such a variable represent the underlying, unrefined value, and we must cast them back into a refined value, as is shown in the rule above right. The rule hypothesis “ x is local” is left imprecise for simplicity and formalized by splitting the type environment into a local and a global part. In our example, the occurrence of n in the body of f is elaborated to $r_n^e \doteq \mathbf{inj}(n_{/(0,L)}, \{n:\mathbf{Int} \mid 8 < n_{/(0,L)}\})$.

Application. To complete the elaboration of our declaration, let us look at the rule for applications.

$$\frac{\Gamma \vdash r_1 \Rightarrow x:R_x \rightarrow R \rightsquigarrow r_1^e \quad \Gamma \vdash r_2 \Leftarrow R_x \rightsquigarrow r_2^e}{\Gamma \vdash r_1 r_2 \Rightarrow R\{x \setminus r_2^e\} \rightsquigarrow r_1^e r_2^e} \text{E-APP}$$

The first thing to notice is that we *check* the type of arguments. Since they are refinements, for which the only typechecking rule is subtyping, there will always be a subsumption. Thus, the expression $r_{xs} \doteq \text{Cons } n \text{ Nil}$ of our example elaborates to: (where $\{\mathbf{IList}\}$ is $\{v:\mathbf{IList} \mid \mathbf{True}\}$)

$$r_{xs}^e \doteq \text{Cons}(\mathbf{sub}(r_n^e, \{n:\mathbf{Int} \mid 8 < n_{/(0,L)}\}, \{n:\mathbf{Int} \mid 5 < n_{/(0,L)}\}))(\mathbf{sub}(\text{Nil}, \{\mathbf{IList}\}, \{\mathbf{IList}\}))$$

Because `Cons` is of type $n:\{n:\mathbf{Int} \mid 5 < n\} \rightarrow \{\mathbf{IList}\} \rightarrow \{\mathbf{IList}\}$. Notice that data constructors are global names, therefore they are considered refined and do not require an injection.

The second thing to notice about the application rule is the substitution of the argument r_2^e inside the refinements of R . This is why the syntax is restricted to only allow refinements as arguments. But this also emphasizes the importance of the projection of dependent variables inside type refinements (performed by the rule E-TFUN). Indeed, the term r_2^e has a refined type (R_x), but must be substituted inside the type refinements in R . But by elaboration, the occurrences of x in R are all projected, and so will be the substituted occurrences of r_2^e .

In our example, this is most visible on the type of the call to `get`, rendered in the subtyping constraints of the elaborated term. Recall the type of `get`: $xs:\{\mathbf{IList}\} \rightarrow i:\{i:\mathbf{Int} \mid 0 \leq i \wedge i <$

```

DtoR(data  $TC := (C_i :: (x_i^j : R_i^j)_{j \leq m_i} \rightarrow \{x : TC \mid r_i\})_{i \leq n} \doteq$ 
  Inductive  $TC_u$ : Set := ( $| C_u^i : \text{TtoU}(R_i^1) \rightarrow \dots \rightarrow \text{TtoU}(R_i^{m_i}) \rightarrow TC_u)_{i \leq n}$ .
  Fixpoint  $\text{wf}_{TC} (x : TC_u)$ : Prop :=
    match  $x$  with ( $| C_u^i x_i^1 \dots x_i^{m_i} \mapsto \text{RtoP}(r_i \sigma) \wedge \bigwedge_{j \leq m_i} \text{RtoP}(r_i^j \sigma)$ )  $_{i \leq n}$  end.
  Global Notation  $TC := \{x : TC_u \mid \text{wf}_{TC} x \wedge \text{True}\}$ .
  (Definition  $C_i (x_i^j : \text{TtoR}(R_i^j; \emptyset))_{j \leq m_i} : \text{TtoR}(R; \emptyset) :=$ 
    exist ( $\lambda x, \text{wf}_{TC} x \wedge \text{True}$ ) ( $C_u^i [x_i^1] \dots [x_i^{m_i}]$ )  $\text{!tac}:(\text{solver}).$ )  $_{i \leq n}$ 
  where  $\{\_ : \_ \mid r_i^j\} = R_i^j$  and  $\sigma = \{\text{proj}(x_i^k) \setminus x_i^k\}_{k \leq m_i}$ .

```

Fig. 2. Translation of an inductive datatype. See table 1 for the definitions of the auxiliary functions.

$\text{llen } xs \rightarrow \{v : \text{Int} \mid 5 < v\}$. The elaboration of $\text{get } r_{xs} \ 0$ is:

$$r_{\text{get}}^e \doteq \text{get}_{/(2,G)} \text{sub}(r_{xs}^e, \{\text{IList}\}, \{\text{IList}\}) \\ \text{sub}(r_0^e, \text{ty}(0), \{i : \text{Int} \mid 0 \leq i_{/(0,L)} \wedge i_{/(0,L)} < \text{llen}_{/(1,G)}(\text{proj}(r_{xs}^e))\})$$

We can see how the term r_{xs}^e , of refined type $\{v : \text{IList} \mid \text{True}\}$, is substituted inside a projection, which is necessary since llen within the context of the type refinement acts on unrefined values.

A last subsumption is necessary to go from the type of the function body $\text{get} (\text{Cons } n \text{ Nil}) \ 0$ to the desired return type, and the complete declaration elaborates to:

$$\text{def } f :: R_f^e := \text{sub}(r_{\text{get}}^e, \{v : \text{Int} \mid 5 < v_{/(0,L)}\}, \{v : \text{Int} \mid v_{/(0,L)} = \text{proj}(n)\})$$

Hints. Our formalization explicitly considers “proof hints” $r ? r_h$, a common technique in Liquid Haskell for adding a fact (r_h) for SMT to use while considering r . The term $r ? r_h$ elaborates to $r ? (r_h \text{ proves } r_p)$, where r_h is the proof term given as hint and r_p is the proposition (the refinement of the type of r_h) that it proves.

$$\frac{\Gamma \vdash r_h \Rightarrow \{x : \text{C} \mid r_p\} \rightsquigarrow r_h^e \quad \Gamma, x : \{x : \text{C} \mid r_p\} \vdash r \Rightarrow R \rightsquigarrow r^e}{\Gamma \vdash r ? r_h \Rightarrow R \rightsquigarrow r^e ? (r_h^e \text{ proves } r_p)} \text{E-HINT}$$

3.3 Top-level Translation of Data Type and Function Declarations

We translate top-level definitions of λ_r , i.e., define datatypes (§ 3.3.1) and functions (§ 3.3.2), using the function DtoR (table 1). Here, we present how DtoR translates each annotated declaration. A program is a sequence of declarations translated homomorphically. Both translations generate additional lemmas and proof automation hints; omitted here but detailed in appendix E.1.

3.3.1 Translation of Inductive Datatype Declarations. The translation of a type constructor TC (fig. 2) outputs four main kinds of Rocq declarations.

First, an unrefined inductive datatype TC_u , using the unrefined translation of types TtoU to generate the plain Rocq types. Second, a predicate wf_{TC} on the unrefined datatype that ensures well-formedness by encoding the original refinements as propositions, using the translation RtoP . We use a substitution σ to replace the projections of the constructor parameters by the variable itself, because in that context the variables are already unrefined. Third, an alias TC for the subset type on TC_u with the well-formedness predicate. This alias has a trivial refinement **True**, to harmonize the notation with further refined versions of TC (this was omitted in the overview § 2).

```

Dtor((reflect)? def f :: Rf := e) ≐
  Definition f (xi : TtoR(Ri; ∅))i≤n : TtoR(R; ∅). destruct (xi as [xi x̂i])i∈I. EtoTac(e) Defined.
  Inductive f_rel : TtoU(R1) → ... → TtoU(Rn) → TtoU(R) → Prop :=
    if reflect then (| fp : pathInd(f, p))p∈paths(e; x1, ..., xn) else ltac : (fun_to_rel f).
  Theorem f_rel_func (xi : TtoU(Ri))i≤n (v v' : TtoU(R)):
    f_rel x1 ... xn v → f_rel x1 ... xn v' → v = v'.
  Theorem f_f_rel (xi : TtoR(Ri; ∅))i≤n (v : TtoU(R)): [f (xi)i≤n] = v ⇔ f_rel ([xi])i≤n v.
  #[global] Instance f_pack : TtoR(Rf; ∅) :=
    { def := f; rel := f_rel; f_f_rel := f_f_rel; func := f_rel_func }.
  #[global] Instance f_upack : TtoU(Rf) := { rel_u := f_rel; func_u := f_rel_func }.
  where Rf = (xi; Ri)i≤n → R, fp is a unique identifier, and I = {i ∈ [1..n] | Ri is not an arrow}.

```

Fig. 3. Translation of a function declaration

Fourth, the translation defines the refined data constructors. The definition of the body casts the unrefined data constructors to satisfy the well-formedness predicate. This time the parameters are refined and must be projected (inside Rocq) to be applied to the unrefined constructor.

The translation also instantiates two equalities (not shown here) for builtin and user-defined, inductive type: `==` (that returns `Prop`) and `==?` (that returns `bool`).

3.3.2 Translation of Function Declarations. The translation of a function f (fig. 3) creates the *refined function* f and the unrefined *graph relation* f_rel . A theorem f_f_rel relates those two versions and the theorem f_rel_func states the functionhood of the relation. These four components instantiate a refined *Pack* instance f_pack (§ 2.6), while the relation and functionhood only instantiate an unrefined *Pack* instance f_upack .

The refined definition f is given by the translation to tactics $EtoTac(e)$. To allow pattern matching, the first-order arguments are destructed into their unrefined value and refinement proof.

For the unrefined graph relation, there are two possibilities. If f is unreflected, we treat it as an uninterpreted function (like Liquid Haskell): we do not allow computation on the relation. The translation builds a generic, opaque graph relation with the custom tactic `fun_to_rel`. If f is reflected, its definition is encoded in the inductive f_rel . Like in Liquid Haskell, reflection of functions with let-bound higher-order variables is unsupported.

The inductive graph relation is generated by collecting all the *function paths* of f , that is, flattening the structure of the function into the constituents of its graph with respect to the branchings in conditionals and case expressions, and excluding the **unreachable** branches. For each path p , we map the unique constructor f^p to the path relation, given by `pathInd(f, p)`, defined in 3.5.

In the statement f_f_rel , notice how we use the projection $\llbracket x_i \rrbracket$ on each parameter x_i . This projection is generic, i.e., x_i can be of a simple refinement type or, if f is higher-order, of a *Pack* type. We define these operators as instances of a typeclass for generic projections.

The type $TtoR(R_f; \emptyset)$ is the type of the *Pack* instance f_pack : our translation transforms arrow types into *Pack* types (see definition 3.6). Arrow types in the output only appear at top-level, as in the type of f . This is also why we do not need to define a projection instance for arrows.

We simplify the presentation of *Packs* in this paper. In reality, to have a generic notion of *Packs*, that support an arbitrary number of arguments and nested higher-order arguments, we encode function arguments inside *Packs* as lists for unrefined arrows, and dependent lists for refined types.

Therefore, the creation of a `Pack` and the projection of an element is done indirectly using tactics, and hides a conversion to and from this encoding.

3.4 Unrefined Translations of Types and Refinements

We present the translation of types and of refinements to the unrefined world. A refinement is translated either as a stand-alone proposition or as a constructor of the graph relation of a function.

Definition 3.1 (Unrefined translation of types). The translation `TtoU` erases refinements of base types, translates inductive types to their unrefined version and creates an unrefined `Pack` type for arrow types. We translate the fixed length `Int` type to `Z`. Overflow and underflow on Haskell’s `Int` operations are undefined behavior, so arbitrary precision arithmetic on `Z` respects these semantics.

$$\begin{aligned} \text{TtoU}(\{x:B \mid r\}) &\doteq \text{Prim}(B) & \text{TtoU}(\{x:TC \mid r\}) &\doteq TC_u \\ \text{TtoU}((x_i:R_i)_{0 \leq i \leq n} \rightarrow R) &\doteq \text{uPack}(\text{TtoU}(R_1), \dots, \text{TtoU}(R_n), \text{TtoU}(R)) \\ \text{where } \text{Prim}(\text{Bool}) &\doteq \text{bool} & \text{Prim}(\text{Int}) &\doteq \text{Z} & \text{Prim}(\text{()}) &\doteq \text{Unit} \end{aligned}$$

3.4.1 Translation of refinements. The translation of refinements to the unrefined world is complicated by graph relations. A function call $f \bar{r}$ must be translated by a call to the relation $f_rel \bar{r} z$, with a fresh variable z standing for the result of $f \bar{r}$. But $f_rel \bar{r} z$ is of kind `Prop`, and as such cannot just replace the original call in the translation. Instead, we store that proposition into a context of hypotheses, and put z in place of $f \bar{r}$. We start by defining a partial translation of refinements `RtoU` that replaces every function call by a fresh variable.

Definition 3.2 (Partial translation of refinements). We assume a map `RV` (for *result variables*) from fully applied functions and operators in λ_r to fresh (`Rocq`) variables.

$$\begin{aligned} \text{RtoU}(x_{/(n,L)}) &\doteq x & \text{RtoU}(x_{/(n>0,G)}) &\doteq x_upack \\ \text{RtoU}(\ell) &\doteq \text{Prim}(\ell) & \text{RtoU}(r_1 = r_2) &\doteq \text{RtoU}(r_1) ==? \text{RtoU}(r_2) \\ \text{RtoU}(C \bar{r}) &\doteq C_u \overline{\text{RtoU}(r)} & \text{RtoU}(\mathbf{proj}(r)) &\doteq \llbracket \text{RtoR}(r) [\emptyset] \rrbracket \\ \text{RtoU}(r) &\doteq \text{RtoU}(r), \text{ if } r \in \{\mathbf{inj}(r, R), \mathbf{sub}(r, R_s, R_t), r ? (r_h \text{ proves } R_p)\} \\ \text{RtoU}(r) &\doteq \text{RV}(r), \text{ if } r \in \{r_1 \odot r_2, \neg r, x_{/(0,G)}, f_{/(n>0,_)} r_1 \dots r_n, \mathbf{proj}(f_{/(n>0,_)} r_1 \dots r_n)\} \end{aligned}$$

where $\text{Prim}(\mathbf{True}) \doteq \mathbf{true}$, $\text{Prim}(\mathbf{False}) \doteq \mathbf{false}$, $\text{Prim}(\text{()}) \doteq \mathbf{tt}$, and $\text{Prim}(\ell) \doteq \ell$ for integers.

Local variables are kept as is; if higher-order, the name x refers to an instance of `uPack`. A non-applied top-level function name x is translated to the name of its unrefined `Pack` x_upack . For equality, we use our boolean equality `==?` (§ 3.3.1). Data constructors are translated to their unrefined versions. A projection `proj`(r) (only occurring in type refinements) tells us to translate r as refined and project the result. Indeed, `Rocq` does a substitution of the refined argument when applying a dependent arrow type (see the example of § 3.2). Injections, subsumptions, and proof hints are ignored since they deal with refinements. Every (fully) applied function call is replaced by the fresh variable given by `RV`. This includes builtin operators, that come with a refined definition and a graph relation in our `Rocq` library, and top-level constants.

3.4.2 Translation of type refinements to propositions. A type refinement r is translated by closing `RtoU`(r) with the hypotheses binding its free variables as results of the functions and operator calls.

Example 3.3. Recall the type of the Liquid Haskell function `get` from the overview (§ 2.5):

$$\text{get} :: xs:\{\text{List}\} \rightarrow i:\{i:\text{Int} \mid 0 \leq i \wedge i < \text{llen}(\mathbf{proj}(xs))\} \rightarrow \{v:\text{Int} \mid 5 < v\}$$

For clarity, we omit the variable annotations but kept the projection, added by elaboration. Suppose $\{(0 \leq i, b_1), (\text{llen}(\mathbf{proj}(xs)), n), (i < n, b_2), (b_1 \wedge b_2, b)\} \subset \text{RV}$. The refinement of i translates to:

$$\forall b_1, \leq_{\text{rel}} 0 \ i \ b_1 \rightarrow \forall n, \text{llen_rel} \llbracket xs \rrbracket n \rightarrow \forall b_2, <_{\text{rel}} i \ n \ b_2 \rightarrow \forall b, \wedge_{\text{rel}} b_1 \ b_2 \ b \rightarrow \mathbf{is_true}(b)$$

Note that the implementation simplifies the output, to obtain the equivalent proposition, presented in the overview: $\leq_{\text{rel}} 0 \ i \ \mathbf{true} \wedge \forall n, \text{llen_rel} \llbracket xs \rrbracket n \rightarrow <_{\text{rel}} i \ n \ \mathbf{true}$.

Given a simple term r , we define the sequence $\text{Hyp}(r)$ that contains all the hypotheses corresponding to the calls of r , in the correct scope order. In example 3.3, there are four such hypotheses. In the case $r_f \doteq f_{/(n,G)} r_1 \dots r_n$ where f is a global variable, Hyp gives the hypothesis $\forall \text{RV}(r_f), f_rel \ \text{RtoU}(r_1) \dots \text{RtoU}(r_n) \ \text{RV}(r_f)$. We have similar hypotheses using the graph relation for top-level constants (when $n = 0$) and builtin operators.

Other than that, for $n > 0$, we extract $r_f \doteq f_{/(n,L)} \bar{r}$ as $\forall \text{RV}(r_f), f.(\text{rel_u}) \overline{\text{RtoU}(r)} \ \text{RV}(r_f)$ and $r_f \doteq \mathbf{proj}(f_{/(n,L)}) \bar{r}$ as $\forall \text{RV}(r_f), f.(\text{rel}) \ \text{RtoU}(r) \ \text{RV}(r_f)$. In both cases, f is a parameter: in the first case, we translate a function body as a constructor of the relation, so f is an unrefined Pack , and in the second case we translate a dependent arrow type, and f is a refined Pack .

Definition 3.4. $\text{RtoP}(r) \doteq h_1 \rightarrow \dots \rightarrow h_n \rightarrow \mathbf{is_true}(\text{RtoU}(r))$, where $(h_1, \dots, h_n) = \text{Hyp}(r)$.

Notice how refinements, that are booleans in Liquid Haskell, are translated to propositions. We use $\mathbf{is_true}$ to cast a boolean to Prop . In Rocq, existential values use Prop because it is proof-irrelevant.

3.4.3 Construction of the graph relation. The unrefined translation RtoU and the hypotheses of Hyp also come together when defining the inductive graph relation from function paths (§ 3.3.2).

A function path for a function is a triple $(\overline{r_{xs}}, \sigma, r)$ containing: the final pattern for its arguments $(\overline{r_{xs}})$, additional matches on function or operator calls (σ) , and a refinement, the expression for that path. The construction of the paths of a function (function paths from § 3.3.2) is given in definition B.2. We reorganize and collapse all pattern matchings and conditionals, and substitute away let-bindings, which is possible since we do not allow binding of higher-order variables. Thus, the final term of a path always ends up being a refinement.

Definition 3.5 (Graph relation of a path). The type of each constructor of the graph relation for a function f is given by pathInd as follows, where in both cases of go , $(h_1, \dots, h_n) \doteq \text{Hyp}(r) \setminus H$.

$$\begin{aligned} \text{pathInd}(f, (\overline{r_{xs}}, \sigma, r)) &\doteq \forall (y)_{y \in \text{fv}(\overline{r_{xs}})}, \text{go}(\sigma, \emptyset) \\ \text{where } \text{go}(\emptyset, H) &\doteq h_1 \rightarrow \dots \rightarrow h_n \rightarrow f_rel \ (\overline{\text{RtoU}(r_{xs})}) \ \text{RtoU}(r) \\ \text{go}((r, r_p) : \sigma, H) &\doteq h_1 \rightarrow \dots \rightarrow h_n \rightarrow \forall (y)_{y \in \text{fv}(r_p)}, \text{RtoU}(r) == \text{RtoU}(r_p) \rightarrow \text{go}(\sigma, H \cup \text{Hyp}(r)) \end{aligned}$$

The type is parametrized by the free variables introduced by the parameter patterns and is inductively defined on the elements of σ . We store already declared hypotheses in a set H to avoid duplicating them. In the auxiliary function, if σ is empty, we declare all the remaining hypotheses of r and return the graph relation of the function f_rel with inputs the destructions of the arguments and result the translation of r . Otherwise, there is a match (r, r_p) , so we declare all the novel hypotheses of r (r_p has none as it is a pattern), declare the match as an equality (using the equality $==$ from § 3.3.1) and continue.

3.5 Refined Translations of Types, Refinements, and Expressions

We present the translation of types, refinements, and expressions into the refined world. These functions take have an additional parameter xs used in the translation of recursive calls and updated with pattern matching. It contains the sequence of patterns for the parameters of the function under translation. To make the notation lighter, we only write down this argument when it is created (an

empty list in figs. 2 and 3), or used and modified (in definitions 3.7 and 3.8). The functions RtoU and RtoP also have this additional parameter, but only pass it down.

Definition 3.6 (Refined translation of types). The function TtoR(R) turns refined types into subset types by translating refinements to propositions. Prim(B) is given in definition 3.1. The translation of a refined type constructor adds the well-formedness condition wf_{TC} encoding the refinements of the original constructor. Arrow types become Packs signatures, similar to the unrefined translation.

$$\begin{aligned} \text{TtoR}(\{x:B \mid r\}) &\doteq \{x:\text{Prim}(B) \mid \text{RtoP}(r)\} \\ \text{TtoR}(\{x:TC \mid r\}) &\doteq \{x:TC_u \mid \text{wf}_{TC} x \wedge \text{RtoP}(r)\} \\ \text{TtoR}((x_i:R_i)_{i \leq n} \rightarrow R) &\doteq \text{Pack}(\forall(x_i:\text{TtoR}(R_i))_{i \leq n}, \text{TtoR}(R)) \end{aligned}$$

3.5.1 Translation of expressions and refinements. The refined translations of expressions and refinements are used to translate function bodies, translating structural expressions into tactics (see § 2.3) and refinements (used within expressions) into terms with casts.

Definition 3.7 (Refined translation of expressions). Function EtoTac is defined as follows:

$$\begin{aligned} \text{EtoTac}(r) &\doteq \text{exact}(\text{RtoR}(r)) & \text{EtoTac}(\text{unreachable}) &\doteq \text{exfalse}; \text{ltac}:(\text{solver}). \\ \text{EtoTac}(\text{if } r \text{ then } e_1 \text{ else } e_2) &\doteq \text{destruct } \llbracket \text{RtoR}(r) \rrbracket \text{ eqn}?: \{ \text{EtoTac}(e_i) \}_{i=1,2} \\ \text{EtoTac}(\text{let } x :: R := e_1 \text{ in } e_2) &\doteq & (\text{arity}(R) = 0) \\ &\quad \text{assert } (x': \text{TtoR}(R)). \{ \text{EtoTac}(e_1) \} \text{ destruct } x' \text{ as } [x \hat{x}]. \text{EtoTac}(e_2) \\ \text{EtoTac}(\text{let } x :: R_x \doteq (x_i:R_i)_{i \leq n} \rightarrow R := e_1 \text{ in } e_2) &\doteq & (\text{arity}(R_x) > 0) \\ &\quad \text{assert } (x': \forall(x_i:\text{TtoR}(R_i))_{i \leq n}, \text{TtoR}(R)). \{ \text{intros } (des_i)_{i \leq n}. \text{EtoTac}(e_1) \} \\ &\quad \text{unshelve refine (let } x:\text{TtoR}(R_x) := \text{ltac}:(\text{fun_to_pack } x') \text{ in } _). \\ &\quad \text{EtoTac}(e_2) & \text{where } des_i = x_i \text{ if } R_i \text{ is an arrow, otherwise } des_i = [x_i \hat{x}_i] \\ \text{EtoTac}(\text{case } r \text{ of } (C_i \bar{y}_i \Rightarrow e_i)_{i \leq n}; xs) &\doteq \\ \left\{ \begin{array}{ll} (\text{generalize dependent } z;)_{z \in (xs \setminus \{x\}) \cap \mathcal{V}} \text{ induction } x \text{ as } [des_1 \mid \dots \mid des_n]; \text{intros.} \\ \{ \text{EtoTac}(e_i; xs\{x \leftarrow C_i \bar{y}_i\}) \}_{i \leq n} & (r = x \in xs \cap \mathcal{V}) \\ \text{destruct } \llbracket \text{RtoR}(r; xs) \rrbracket \text{ as } [\bar{y}_1 \mid \dots \mid \bar{y}_n] \text{ eqn}?: \{ \text{EtoTac}(e_i; xs) \}_{i \leq n} & (r \notin xs \cap \mathcal{V}) \end{array} \right. \end{aligned}$$

EtoTac inserts the translation of refinements as terms in a proof script with the tactic exact.

For a let-binding, EtoTac adds a new variable to the context with assert t. It creates an preliminary variable x' binding the translation of e_1 . If x is first-order, x' is a dependent pair that is splitted into the unrefined value x and the witness. Otherwise, x' is a refined function. Then, x is a Pack created from x' and containing the generic graph relation used for unreflected functions.

Pattern matching on a refinement r can be translated in two ways. If r is one of the parameters of the function ($r = x \in xs \cap \mathcal{V}$), induction creates the relevant induction hypothesis. For each $i \leq n$, des_i is the sequence $(y_i^j)_{j \leq m_i}$ with names $\text{IH}y_i^j$ inserted if y_i^j is an inductive constructor and if $\text{IH}y_i^j$ is ever used in the translation of e_i . If y_i^j is an inductive constructor but the induction hypothesis is not used, a hole is put instead of $\text{IH}y_i^j$ to avoid creating the hypothesis.

In the branches, EtoTac updates the parameter xs by replacing the matched variable x by the corresponding pattern. The other parameters are generalized before induction, which avoids capturing them in the created induction hypotheses, so that they can be later instantiated correctly.

Since we only support structural recursion, the recursive calls always come from the destruction of a parameter. If the matched is not on a parameter, there is no need to create induction hypotheses. EtoTac calls destruct for a simple case analysis. The additional eqn:? asserts the equality between the term and its pattern in the context of each branch. Ifs are also translated with destruct.

All missing branches are translated with exfalse that asks for a proof of **False** in the branch context. This is consistent with Liquid Haskell that requires totality under the refinements.

Definition 3.8 (Refined translation of refinements). Function RtoR is defined as follows:

$$\begin{aligned}
 \text{RtoR}(\ell) &\doteq \text{Prim}(\ell) & \text{RtoR}(C) &\doteq C \\
 \text{RtoR}(x_{/(n>0,G)}) &\doteq x_pack & \text{RtoR}(x_{/(_,_)} &\doteq x \\
 \text{RtoR}(\neg r) &\doteq \neg \text{RtoR}(r) & \text{RtoR}(r_1 \odot r_2) &\doteq \text{RtoR}(r_1) \odot \text{RtoR}(r_2) \\
 \text{RtoR}(r_1 = r_2) &\doteq \text{RtoR}(r_1) ==? \text{RtoR}(r_2) \\
 \text{RtoR}(\mathbf{sub}(r, R_s, R_t)) &\doteq \text{TtoR}(R_t) \leftarrow_{\text{TtoR}(R_s)} \text{RtoR}(r) \\
 \text{RtoR}(\mathbf{inj}(r, R)) &\doteq \text{exist } (\forall (x:\mathbb{U}), \mathbb{Q}) \text{RtoR}(r) \text{ ltac: (solver), where } \{x:\mathbb{U} \mid \mathbb{Q}\} = \text{TtoR}(R) \\
 \text{RtoR}(r ? (r_h \text{ proves } r_p)) &\doteq \text{let } (_ : \text{RtoP}(r_p)) := \text{proj2_sig}(\text{RtoR}(r_h)) \text{ in } \text{RtoR}(r) \\
 \text{RtoR}(r \ r_1 \dots r_n; xs) &\doteq \begin{cases} \text{IH}y \text{ ltac: (solver)} (\mathbb{P}i)_{\mathbb{P}i \in \text{arg}_i, i \in [1..n] \setminus k} & \text{if } r = f_{/(n,Y)} \\ f.(\text{def}) \text{RtoR}(r_1; xs) \dots \text{RtoR}(r_n; xs) & \text{if } r = f_{/(n>0,L)} \\ \text{RtoR}(r; xs) \text{RtoR}(r_1; xs) \dots \text{RtoR}(r_n; xs) & \text{otherwise} \end{cases}
 \end{aligned}$$

Where in the last equations n is maximal and in first of the three cases, with $xs \doteq (r_{xs}^1, \dots, r_{xs}^n)$:

- $I \doteq \{i \mid r_{xs}^i \in \mathcal{V}\}$; y and $1 \leq k \leq n$ are such that $r_k = y$ and $y \in \text{fv}(r_{xs}^k)$ but $r_{xs}^k \neq y$;
- $\text{arg}_i = \text{RtoR}(r_i; xs)$ if r_i is higher-order and else $\text{arg}_i = (\llbracket \text{RtoR}(r_i; xs) \rrbracket, \text{ltac: (solver)})$.

The translation of a literal is a builtin literal of Rocq. They have a base type (RtoR uses the same function $\text{Prim}(\ell)$ as RtoU), but are always surrounded by an injection to their singleton type, as explained in § 3.2. Data constructors, on the other hand, are the refined ones from fig. 2. Operators are translated to the custom operators with refined types that we define in our Rocq library (and for which we use here the same symbols). Equality however is translated to the equality operator from our equality typeclass, which is defined for subset types as the equality of the projected terms.

Variables names, when higher-order, must always refer to a Pack instance. The translation of a top-level function name x is thus its instance x_pack , similar to what happens in RtoU . If x is local, it is already a Pack by construction. An injection is translated as the construction of a dependent pair from the translation of the term and a proof generated by solver.

RtoR translates a proof hint as a let-binding in order to put the proposition r_p in the context, then asserts it using its proof r_h . Rocq's operator proj2_sig retrieves the second element of the dependent pair and the placeholder lets Rocq infer a fresh variable name.

The translation of an application is subtle. A non-recursive call (second and third case) is translated recursively on the subterms. If the head term is a local variable f (second case), then f binds a Pack, so RtoR projects on the field containing the actual refined function.

A recursive call (first case) is translated as a call to the induction hypothesis $\text{IH}y$ obtained from the translation of pattern matching. Since we allow only structural recursion on the arguments, in a recursive call $f \ r_1 \dots r_n$ there is one of the arguments r_k that is an inductive variable y introduced by the destruction of a parameter. There can be several of those, in which case we arbitrarily select the first one from left-to-right in the order of arguments in the implementation.

The induction hypothesis $\text{IH}y$ is already specialized to y . It is applied to the projection of the translation of the remaining arguments that have not been already destructed (set I). The induction hypothesis takes the value and refinement (given by solver) of first-order arguments separately. The first call to solver is for the refinement of x itself.

3.5.2 Subsumption casts. In definition 3.8, we use the notation $R_t \leftarrow_{R_s} e$ to cast the Rocq term e of source type R_s to the target type R_t . Such subsumption casts are defined both for subset types (over the same base type) and Packs. In the case of subset types, we define:

$$\{x_t:\mathbb{U} \mid \mathbb{P}_t\} \leftarrow_{\{x_s:\mathbb{U} \mid \mathbb{P}_s\}} e \doteq \text{exist } (\forall (x_t:\mathbb{U}), \mathbb{P}_t) \llbracket e \rrbracket \text{ ltac: (pose proj2_sig(e); solver)}$$

The cast projects $\mathbb{e}$, then injects it back to the expected refined type. The solver proves the verification condition: assuming the refinement of $\mathbb{e}$ ($\text{proj2_sig}(\mathbb{e})$), it creates a proof of $\mathbb{P}_t\{x_t \setminus \llbracket \mathbb{e} \rrbracket\}$.

Subsumption on Packs separately casts the four elements of the Pack: two unrefined (the graph relation and its functionhood lemma) and two refined (the definition and the lemma relating the definition and the graph relation). The first two are unaffected by the cast, being unrefined. The lemma relating the definition and the graph relation is proven again using the original lemma. The refined definition is morally translated by casting the arguments contravariantly and the function body covariantly (*per* [30]). The actual implementation, described in appendix D, is more complex because of the representation of function arguments as dependent lists inside Packs (§ 3.3.2).

4 Properties of the Translation

Our translation is sound, in the sense that it maps a proof of falsity in Liquid Haskell to a proof of falsity in Rocq. That is, Lava lets Liquid Haskell use Rocq as a proof checker, via translation. This check increases our confidence in Liquid Haskell, but our tool is not complete in the sense that a well typed Liquid Haskell program necessarily yields a well-typed Rocq program with provable proof obligations that solver can resolve. Such a proof would have to relate an SMT solver to our translation’s encoding; a very ambitious goal! Instead, we evaluate empirically (§ 6).

Preservation of evaluation semantics of expressions. To show that proofs of false are preserved, we first show that our translation preserves the input/output behavior of declared functions—both in the graph relations of the unrefined world and in the tactic-generated executable terms of the refined world. We introduce a conventional call-by-value reduction relation $\rightarrow_{\text{er}}$ (definition C.1) for expressions as a small-step operational semantics, parametrized by a sequence of declarations $\bar{D}$.

THEOREM 4.1. *Let $\bar{D}$ be a well-formed and annotated sequence of declarations, with all functions reflected. Let **reflect** **def** $f :: (x_i : R_i)_{i \leq n} \rightarrow R := e_f \in \bar{D}, v_1, \dots, v_n, v$ be values (variables, literals and constructors) such that $\bar{D} \vdash f v_1 \dots v_n \rightarrow_{\text{er}}^* v$. Then, in Rocq and under $\text{DtoR}(\bar{D})$:*

(i) $f_rel \text{ RtoU}(v_1) \dots \text{RtoU}(v_n) \text{ RtoU}(v)$ holds and (ii) $\llbracket \text{RtoR}(f v_1 \dots v_n) \rrbracket[(x_i)_{i \leq n}] = \text{RtoU}(v)$.

PROOF SKETCH. We prove (i) by induction. We relate the graph relation and the refined translation of a function (ii) directly in Rocq with our lemma f_f_rel (§ 3.3.2). Since we assume that the Rocq output typechecks, the property (ii) is verified. The full proof is in appendix C.1. $\square$

We restrict this theorem to the case where all functions are reflected (any function can be reflected so long as it does not let-bind higher-order variables). For unreflected functions, the graph relation is defined in terms of the refined function, which is defined with tactics. Proving anything about the output of these tactics is quite challenging.

Preservation of the models of types. We show that our translation preserves the semantics of types. To do so, we interpret both λ_r and Rocq types in the same model and show that the translation preserves the type’s model. For Rocq types, we only need to define models for types in the image of the translation, that is, instances of the typeclass Pack, as well as refinement (or subset) types of the form $\forall(x : \$), \$$ and $\{x : \mathbb{U} \mid \mathbb{Q}\}$, where $\mathbb{U}$ is a base type and $\mathbb{Q}$ a predicate coming from the translation of a refinement as in definition 3.4; we have $\mathbb{Q} \Rightarrow \text{wf}_{TC} x$ if $\mathbb{U} = TC_u$.

We give the standard interpretation for refinement types [24, § 5]. We interpret them as the set of terms t such that t is well-typed in the underlying simply-typed system, and in the case of a refined base type with refinement r or $\mathbb{Q}$, such that r or $\mathbb{Q}$ instantiated by t is true.

We write $\llbracket \cdot \rrbracket_\rho^\Gamma$ for the interpretation of λ_r types and $\llbracket \cdot \rrbracket_\rho^\Gamma$ for the interpretation of Rocq types. The typing environment Γ gives type constructor specifications and the assignment ρ maps variables to λ -calculus values. The complete definitions and proofs are in appendix C.2.

Table 2. Solving and preprocessing tactics. Higher level tactics use lower level ones.

	Solving Tactic	Preprocessing Tactic	Solver Definition
Level 1	fast_solve	idtac	fast_done
Level 2	quick_solve	quick_simpl	solve repeat solving_loop
Level 3	oracle	cleanup	quick_solve
Level 4	solver	preprocessor	first [quick_solve shelve]

THEOREM 4.2. *Let R be a refinement type in λ_r and Γ a λ_r context of datatypes such that R is well-formed under Γ . For any assignment ρ , $\llbracket R \rrbracket_\rho^\Gamma = \langle \text{TtoR}(R) \rangle_\rho^\Gamma$.*

Theorems 4.1 and 4.2 state that the translation preserves the input-output relation at the level of values and at the level of types (including refinements). In particular, Liquid Haskell proofs are functions of unit type whose output is trivial, but the statements proved will be equivalent in Rocq.

Unprovability of contradiction. Finally, we confirm that our translation can act as a certificate generator for Liquid Haskell. If we manage to prove false in Liquid Haskell, we want to make sure that typechecking will not go through in the Rocq output (assuming Rocq’s consistency).

THEOREM 4.3 (CONSISTENCY). $\nexists e. \emptyset \models_{\text{Rocq}} \text{DtoR}(\text{def } f :: \{x:() \mid \mathbf{False}\} := e).$

PROOF. Suppose there is a well-formedness derivation for f (in the empty context). Then, e can be translated into a definition of type $\{x:() \mid \mathbf{False}\}$. This would give rise to a proof of **False**—a contradiction, because Rocq’s type theory is consistent. $\square$

5 Implementation & the Proof Automation Tactics

We implemented Lava following the translation (§ 3) in ~5K SLOC of Haskell. Lava takes as input a program typechecked by Liquid Haskell and outputs a Rocq file that depends on a) a Rocq library containing, for example, definitions for the Pack typeclasses (~350 SLOC) and b) proof automation tactics (~6K SLOC). In the rest of the section, we describe the development of the tactics.

The solver tactic is the main tactic used by Lava to resolve the proof holes generated by the translation. Table 2 describes the implementation of solver in four levels; each level composes a *preprocessor tactic* and a solver definition into a *solving tactic*. Lower level correspond to lower complexity, with tactics of higher level subsuming and making use of lower level ones.

The *preprocessor tactics* try to simplify proof obligations, only succeeding if provability of the goals is preserved by the transformations. The simpler kind unfolds, applies, and rewrites by lemmas and definitions from the library or the generated files. The more complex kind creates and discards subgoals, relying on solving tactics from lower levels. The preprocessor tactics listed in table 2 combine many simpler preprocessor tactics, each corresponding to a single reasoning step; cleanup and preprocessor can invoke dozens of smaller steps in each successful call.

The *solving tactics* fail if they are unable to discard the goal. They culminate in solver, which uses most of the other definitions and tactics in the library. Unlike the other solving tactics, solver shelves subgoals it cannot prove, which is useful to inspect proof scripts manually. The class of solving tactics also contains numerous tactics for proving the lemmas generated by the translation; they reuse many of the definitions also used by solver.

Let us give more details on each individual level.

On level 1, *idtac* is a noop; *fast_done* uses *lia* and *auto* (with a hint library).

On level 2, *quick_simpl* uses simple rewriting lemmas to clean up the goal. The *solving_loop* tactic mostly relies on rewriting, unfolding and solving with hints, simplifying with basic Rocq

tactics (e.g., `simpl`, `split`). It runs rule-based simplification steps (1-5 SLOC each) to simplify specific syntactic forms like quantifiers, logical connectives, or equalities.

On *level 3*, `cleanup` uses `quick_simpl` and lemmas relating refined functions and their graph relations. It works extensively with graph relations, by mapping refined terms to their corresponding graph terms, reasoning about result variables, and saturating the context by reconstructing the refinement witnesses of the final outputs of graph relations.

On *level 4*, preprocessor extends `cleanup` with extra simplification steps which can require proving difficult subgoals with the `oracle` tactic.

Besides solver, the most powerful but expensive (by verification time in our examples) tactics are the ones relating a reflected definition f to its graph relation f_rel , in particular in proving the theorems f_f_rel (§ 3.3.2). These tactics employ many preprocessing steps similar to those of `cleanup`. However, the proofs require additional and more aggressive preprocessing steps, in order to instantiate induction hypothesis and unify them with goals. Such steps apply a forward reasoning that might in general get us stuck in the resolution, but are safe in this context.

Could we replace our automation tactics with existing SMT or other Rocq automation? It would not be so easy: (1) a lot of preprocessing is required to bring the proof state into a shape well supported by Rocq and the tools in its ecosystem; (2) we need to interleave forward and backwards reasoning steps matching on and modifying both goals and hypothesis; and (3) the solving process can be very slow (see the benchmarks in table 3), so customizing automation enables us to optimize performance for our goals. We discuss the use of Sniper and CoqHammer specifically in § 7.

6 Evaluation

We evaluate Lava on eight benchmarks, summarized on table 3. The first five are designed to test specific features of the translation and the other three are ported from standard examples of Rocq (namely, software foundations [27]) or Liquid Haskell (namely, the benchmarks of [40]).

Description of Examples. The Overview example (§ 2) tests most features of Lava. It contains a non-trivially refined inductive data type, a recursive function, one with preconditions, and a reflected function that is translated using graph relation. It also tests the translation of subtyping to casts in Rocq, the translation of Haskell’s integer types to Rocq’s $\mathbb{Z}$ and of a higher-order function.

PeanoNats evaluates the translation of theorems to Rocq. It defines the inductive data type of Peano naturals and reflected functions for addition, multiplication, subtraction, greater than or equal, and equality. It also contains theorems stating that zero is the left and right neutral element of addition, associativity of addition, distributivity of right-multiplication with addition, and that $\forall m, n : (m + n) - n = m$. This example pushes the limits of our proof automation in Rocq.

RBinsToBins tests the translation of a refined datatype. It defines the refined data type of binary numbers without leading zeros and a function translating from it to an unrefined version of the data type. Crucially, the refinement of one of this datatype’s constructors depends on another constructor. Supporting this example motivates our translation of refined datatypes (from § 2.2).

HigherOrder tests the translation of λ -abstractions and properties of function composition.

List tests the list operations `append`, `map`, `concatMap`, `reverse`, `zip`, `flip`, `zipWith`, `unzip` and `take`, and various theorems relating them.

SoftwareFnd is a Liquid Haskell port of the first section of Software Foundations [27] and the first five definitions/theorems of the second and third sections each. The original Rocq file has 356 SLOC, translated to 437 SLOC of Liquid Haskell.

FoldrUniversal encodes the universality of the `foldr` function, as described in [23].

ClassLaws tests the monad and monoid laws on *List*, *Maybe*, and *Identity* datatypes, monomorphized for integers. The last two examples are taken from the benchmarks of [40]. Their complete

Example	LH SLOC	Rocq SLOC	Blowup	Translation Time	Verification Time	Verification Time/SLOC
Overview	36	451	12.5×	0.13 sec	31 min 43 sec	4.22
PeanoNats	88	956	10.9×	0.10 sec	14 min 14 sec	0.89
RBinsToBins	23	148	6.4×	0.13 sec	4 sec	0.03
HigherOrder	22	110	5.0×	0.20 sec	15 sec	0.14
List	178	1 776	10.0×	0.42 sec	12 min 3 sec	0.41
SoftwareFnd	437	5 149	11.8×	0.18 sec	7 min 50 sec	0.09
ClassLaws	151	1 244	8.2×	0.71 sec	2 min 03 sec	0.1
FoldrUniversal	37	237	6.4×	0.18 sec	1 min 37 sec	0.41
Total	972	10 071	10.4×	2.05 sec	69 min 49 sec	0.42

Table 3. Quantitative Evaluation of Lava. SLOC is semantic lines of code. Blowup is the ratio between the LH and the generated Rocq. Verification time is the time to verify the Rocq code. Verification time/SLOC is the verification time (in seconds) divided by the number of Rocq SLOC.

benchmarks suite also contain class laws for the functor and applicative classes that use features, such as partial applications and higher-order datatypes, that are unsupported by the translation.

Summary. We find a 10.1× blowup in code size when moving from Liquid Haskell to Rocq, with fast translation times (<1s) but verification times of up to 32 minutes (table 3). Our translation is not designed to generate concise Rocq code, but is nonetheless rather verbose.

Verification times *per line of Rocq code* vary widely ranging from 4.22 for Overview to 0.09 for SoftwareFnd. We find that *most* of the verification is immediate, but a few hard proof obligations take minutes. Indeed, around 39% of the total verification time (27 min 9 sec) comes from a single solver call proving the final proof obligation in the last theorem in Overview.

So: verification time hugely depends on the difficulty of the proof obligations. That our automation frequently solves hard goals demonstrates their power, but that they take several minutes shows there is room for further optimization. Finally, for hard proof obligations most of the verification time stems from backtracking. In those cases, interactively proving the tricky steps (after preprocessing with our automation tactics) can substantially reduce verification time in practice.

7 Related Work

Translations from Haskell to Rocq. `hs-to-coq` [12, 35] translates Haskell programs to Rocq and is used to verify a substantial part of Haskell’s containers library. This tool differs from ours in two main aspects. First, source programs lack refinement types—and so have neither dependent pattern matching nor proofs terms, simplifying the translation. The desired specifications are either inserted manually or derived from QuickCheck annotations and proved by hand. Second, Breitner et al. support syntactic partiality of programs by inserting an opaque default value. While we allow Haskell programs to have missing cases, we require that Liquid Haskell programs are in fact total.

Our choice to use inductive predicates has its downsides: it makes support for partial applications difficult and forces the extraction of applications to hypotheses in the unrefined translation. Another way to deal with partiality in the input program is to use a monadic translation, like Dylus et al. [18] and Christiansen et al. [15] do for translating Haskell (inspired from [1]), or Claret [16, Chapter 3] and Garrigue and Saikawa [20] do for OCaml. Monads could be used to deal with partial Liquid Haskell programs, that are partial also in the refined world. This is not a priority because the default semantics of Liquid Haskell rejects non-provably total programs.

Translation of programs with refinements in interactive theorem provers. *Stainless* [22] is a refinement type checker for Scala. *Stainless* has a prototype translation to Rocq [19] that is not (yet)

covered in a scientific paper. But while Stainless internally uses refinement types, this translation does not support them nor dependent arrows. In Stainless, refinements are written as pre- and post-conditions and assertions on functions, and this is the specification that the translation supports. The translated programs are effectively unrefined programs with assertions. The primary focus of our work instead is to describe the subtle interaction between unrefined and refined elements, in particular in the context of the embedding of programs into the logic. Thus, their embedding is simpler as everything is unrefined. Despite this, the translation of Stainless’ programs is intended as a backend for verification, much like Lava serves as a certificate checker for Liquid Haskell.

Characteristic formulae are another approach of translating from general-purpose proof assistants to logic: Charguéraud [13] translates OCaml programs to logical formulas in Rocq, instead of Rocq programs. It would be interesting to study similar approaches in future work, in particular also whether one can translate from GOSPEL-annotated OCaml programs [14] using our translation.

Translations of refined inductive datatypes. Atkey, Johann, and Ghani [4] define a systematic translation between refined and inductive datatypes via categorical logic. This differs from our approach of using unrefined inductive datatypes and inductively-defined predicates instead. Sekiyama, Nishida, and Igarashi [31] compare the representations of refined inductive datatypes with unrefined constructors and unrefined inductive types with refined constructors in Rocq. We use the same correspondence to translate inductive datatypes with refined constructors from Liquid Haskell to inductive datatypes refined by a predicate in Rocq.

Mechanization of Liquid Haskell. We aim to improve foundations of Liquid Haskell by producing certificates for programs, using Rocq for a semantics-in-translation. With a similar goal, Borkowski, Vazou, and Jhala [10] define and formalize a type system that models Liquid Haskell (subtyping and type inference algorithm excluded) and prove it sound in Rocq. Writing and verifying the entirety of Liquid Haskell in Rocq is out of reach, in particular as Liquid Haskell is in constant development and a platform for testing research ideas. But ideally their approach and ours would converge: the type system at the root of Lava could be mechanized and proved sound in Rocq.

Explicit refinement types. Ghalayini and Krishnaswami [21] introduce a calculus (λ_{ert}) that makes refinement reasoning explicit by internalizing refinements directly in the core calculus: refinement proofs are first-class proof terms, and typing does not rely on subtyping. Lava shares the goal of making refinement reasoning explicit, but differs fundamentally in approach. Rather than designing a refinement calculus inside type theory, Lava starts from Liquid Haskell’s SMT-based system and translates it to Rocq, reifying implicit subtyping and proof obligations as explicit casts and obligations in the target. Moreover, λ_{ert} operates in a single semantic layer, whereas Lava must model the separation between refined and unrefined worlds that arises in Liquid Haskell. Capturing this intertwined semantics is central to our contribution and does not arise in λ_{ert} .

Programming with dependent types in Rocq. Equations Sozeau and Mangin [33] is a Rocq extension that simplifies writing functions defined by dependent pattern matching. Equations accepts incomplete definitions with holes, and presents them to the user as explicit proof obligations. We could have used Equations to define the refined functions and would have possibly led to more straightforward definitions. This however would still necessitate defining the automation machinery for relating the refined definition and the unrefined graph relation, one of Lava’s more complex components. We decided against using Equations because we expected using tactics to be more predictable. Since Equations offers a handy interface for well-founded recursion, we plan on re-investigating its adoption into Lava for a future extension towards non-structural recursion.

Another possibility for writing refined functions in Rocq is the *Program* extension by Sozeau [32]. Program offers implicit predicate subtyping on subset types. This generally makes generated

code look a lot like Liquid Haskell code, with no explicit casts, but with less automation and proofs to be resolved by the user. We do not target Program as our goal is exactly to make the relation between the unrefined and refined program explicit, through casts and subsumption in particular.

Integrating SMT solvers and hammers into Rocq. The *SMTCoq* project [2] adds SMT support to Rocq (currently CVC4 [5] and veriT [11]), by checking and reconstructing the solver’s proof witnesses back into Rocq. Using *SMTCoq* directly to solve our proof obligations is hard, as a lot of pre-processing must be done. Fortunately, *Sniper* [8, 9] extends *SMTCoq* with many useful transformations of the Rocq proof state. Even though those transformations are designed to be compositional, safe and, certified, we experienced that the tool is not mature enough to be used as a drop-in replacement for our automation tactic.

Overall, it is hard to know when snipe will succeed or not without deep knowledge of the internals of *Sniper*. Subset types and projections often make the tactic fail or loop on simple obligations. On success, *snipe* is always slower than our automation tactic on our examples. This might be improved by selecting and arranging the transformations as is most relevant to us. A lot of transformations are also peculiar to our translation, for instance saturating the context with the hypotheses obtained from the graph relations. Adopting a structured approach to our automation in the style or combined with *Sniper* is an ambitious but interesting project.

Using an SMT solver for the resolution of verification constraints means more hope that a verified property of Liquid Haskell is automatically provable in Rocq. But this also means generating more definitions and proofs, for instance to verify properties of the encoding of datatypes in the solver logic, or translating every inductive proposition to a recursive function in `bool`. *Sniper* gives a way to generate the latter boolean fixpoint automatically, but we often failed to make generation work, or sometimes found the generated fixpoint to be wrong.

Czajka and Kaliszyk [17] define a *hammer* [7] for Rocq. It uses a heuristic translation into a fragment of first-order logic, applies HOL ATPs (like E, Vampire or Z3), then tries to reconstruct the proof into Rocq proof terms. Their translation and proof reconstruction is neither sound nor complete, but the approach works well enough in practice. Lava requires predictable proof automation, since it does not rely on interactive proof development. For this reason we cannot use *CoqHammer* or machine learning-based tools like *Tactician* [6].

8 Conclusion

Lava translates from Liquid Haskell to Rocq, formally relating SMT-based refinement typing and foundational type theory. Lava turns implicit subtyping explicit through casts; it models both semantics—refined and unrefined— and generates (largely automatically solvable) proof obligations in Rocq. Our translation provides a new foundation for Liquid Haskell that correctly accounts for both refined and unrefined semantics in its SMT encoding. Our implementation shows that this translation is practically possible. Polymorphism and other more advanced features merely demand refactoring and engineering; better automation (and automatic solving use SMT/hammers) is a question of connecting the dots. With Lava, we aim to put Liquid Haskell and refinement typing on a more solid footing on its own and in relation to type theoretic approaches—while also helping the community understand the subtleties of refinement types.

References

- [1] Andreas Abel, Marcin Benke, Ana Bove, John Hughes, and Ulf Norell. 2005. Verifying Haskell Programs Using Constructive Type Theory. In *Proceedings of the 2005 ACM SIGPLAN workshop on Haskell (2005-09) (Haskell05)*. ACM, 62–73. doi:10.1145/1088348.1088355
- [2] Michael Armand, Germain Faure, Benjamin Grégoire, Chantal Keller, Laurent Théry, and Benjamin Werner. 2011. A Modular Integration of SAT/SMT Solvers to Coq through Proof Witnesses. In *Lecture Notes in Computer Science* (2011).

- Springer Berlin Heidelberg, 135–150. doi:10.1007/978-3-642-25379-9_12
- [3] Robert Atkey, Patricia Johann, and Neil Ghani. 2011. When Is a Type Refinement an Inductive Type?. In *Foundations of Software Science and Computational Structures* (2011) (*Lecture Notes in Computer Science*, Vol. 6604), Martin Hofmann (Ed.), Springer Berlin Heidelberg, 72–87. doi:10.1007/978-3-642-19805-2_6
 - [4] Robert Atkey, Patricia Johann, and Neil Ghani. 2012. Refining Inductive Types. *Logical Methods in Computer Science* Volume 8, Issue 2 (June 2012). doi:10.2168/lmcs-8(2:9)2012
 - [5] Clark Barrett, Christopher L. Conway, Morgan Deters, Liana Hadarean, Dejan Jovanović, Tim King, Andrew Reynolds, and Cesare Tinelli. 2011. CVC4. In *Computer Aided Verification* (2011). Springer Berlin Heidelberg, 171–177. doi:10.1007/978-3-642-22110-1_14
 - [6] Lasse Blaauwbroek, Josef Urban, and Herman Geuvers. 2020. *The Tactician: A Seamless, Interactive Tactic Learner and Prover for Coq*. Springer International Publishing, 271–277. doi:10.1007/978-3-030-53518-6_17
 - [7] Jasmin C. Blanchette, Cezary Kaliszyk, Lawrence C. Paulson, and Josef Urban. 2016. Hammering Towards QED. *Journal of Formalized Reasoning* Vol 9 (2016), No 1 (2016): Special Issue: Twenty Years of the QED Manifesto. doi:10.6092/ISSN.1972-5787/4593
 - [8] Valentin Blot, Denis Cousineau, Enzo Crance, Louise Dubois de Prisque, Chantal Keller, Assia Mahboubi, and Pierre Vial. 2023. Compositional Pre-processing for Automated Reasoning in Dependent Type Theory. In *Proceedings of the 12th ACM SIGPLAN International Conference on Certified Programs and Proofs* (Boston, MA, USA, 2023-01) (*CPP '23*). ACM, New York, NY, USA, 63–77. doi:10.1145/3573105.3575676
 - [9] Valentin Blot, Louise Dubois de Prisque, Chantal Keller, and Pierre Vial. 2021. General Automation in Coq through Modular Transformations. (July 2021). doi:10.48550/arXiv.2107.02353
 - [10] Michael H. Borkowski, Niki Vazou, and Ranjit Jhala. 2024. Mechanizing Refinement Types. 8, POPL (2024), 2099–2128. doi:10.1145/3632912
 - [11] Thomas Bouton, Diego Caminha B. de Oliveira, David Déharbe, and Pascal Fontaine. 2009. veriT: An Open, Trustable and Efficient SMT-Solver. In *Automated Deduction – CADE-22* (2009). Springer Berlin Heidelberg, 151–156. doi:10.1007/978-3-642-02959-2_12
 - [12] Joachim Breitner, Antal Spector-Zabusky, Yao Li, Christine Rizkallah, John Wiegley, and Stephanie Weirich. 2018. Ready, Set, Verify! Applying hs-to-coq to real-world Haskell code. *Proceedings of the ACM on Programming Languages* 2, ICFP, Article 89 (July 2018), 16 pages. doi:10.1145/3236784
 - [13] Arthur Charguéraud. 2010. Program Verification Through Characteristic Formulae. *ACM SIGPLAN Notices* 45, 9 (2010), 321–332. doi:10.1145/1932681.1863590
 - [14] Arthur Charguéraud, Jean-Christophe Filliâtre, Cláudio Lourenço, and Mário Pereira. 2019. GOSPEL - Providing OCaml with a Formal Specification Language. In *Formal Methods - The Next 30 Years - Third World Congress, FM 2019, Porto, Portugal, October 7-11, 2019, Proceedings (Lecture Notes in Computer Science, Vol. 11800)*, Maurice H. ter Beek, Annabelle McIver, and José N. Oliveira (Eds.). Springer, 484–501. doi:10.1007/978-3-030-30942-8_29
 - [15] Jan Christiansen, Sandra Dylus, and Niels Bunkenburg. 2019. Verifying Effectful Haskell Programs in Coq. In *Proceedings of the 12th ACM SIGPLAN International Symposium on Haskell* (2019-08) (*ICFP '19*). ACM, 125–138. doi:10.1145/3331545.3342592
 - [16] Guillaume Claret. 2018. *Program in Coq*. Ph. D. Dissertation. Université Sorbonne Paris Cité. <https://inria.hal.science/tel-01890983v1/>
 - [17] Łukasz Czapka and Cezary Kaliszyk. 2018. Hammer for Coq: Automation for Dependent Type Theory. *Journal of Automated Reasoning* 61, 1–4 (2018), 423–453. doi:10.1007/s10817-018-9458-4
 - [18] Sandra Dylus, Jan Christiansen, and Finn Teegen. 2018. One Monad to Prove Them All. *The Art, Science, and Engineering of Programming* 3, 3 (Feb. 2018). doi:10.22152/programming-journal.org/2019/3/8 arXiv:1805.08059 [cs.PL]
 - [19] EPFL/LARA. 2024. *Translation from Stainless to Coq*. <https://epfl-lara.github.io/stainless/coq.html> Accessed: 2025-07-07.
 - [20] Jacques Garrigue and Takafumi Saikawa. 2022. Validating OCaml Soundness by Translation into Coq. (2022). https://types22.inria.fr/files/2022/06/TYPES_2022_paper_15.pdf Abstract for Types 2022.
 - [21] Jad Elkhaleq Ghalayini and Neel Krishnaswami. 2023. Explicit Refinement Types. In *Proceedings of the ACM on Programming Languages* (2023-08), Vol. 7. ACM, 187–214. doi:10.1145/3607837
 - [22] Jad Hamza, Nicolas Voirol, and Viktor Kunčák. 2019. System FR: formalized foundations for the stainless verifier. 3, OOPSLA (2019), 1–30. doi:10.1145/3360592
 - [23] Graham Hutton. 1999. A Tutorial on the Universality and Expressiveness of Fold. *Journal of Functional Programming* 9 (09 1999). doi:10.1017/S0956796899003500
 - [24] Kenneth Knowles and Cormac Flanagan. 2010. Hybrid Type Checking. *ACM Transactions on Programming Languages and Systems* 32, 2 (2010), 1–34. doi:10.1145/1667048.1667051
 - [25] Nico Lehmann, Adam T. Geller, Niki Vazou, and Ranjit Jhala. 2023. Flux: Liquid Types for Rust. *Proc. ACM Program. Lang.* 7, PLDI, Article 169 (June 2023), 25 pages. doi:10.1145/3591283

- [26] Xinming Ou, Gang Tan, Yitzhak Mandelbaum, and David Walker. 2004. Dynamic Typing with Dependent Types. In *Exploring New Frontiers of Theoretical Informatics*, Jean-Jacques Levy, Ernst W. Mayr, and John C. Mitchell (Eds.). Kluwer Academic Publishers, Boston, MA, 437–450. doi:10.1007/1-4020-8141-3_34
- [27] Benjamin C. Pierce, Arthur Azevedo de Amorim, Chris Casinghino, Marco Gaboardi, Michael Greenberg, Catalin Hrițcu, Vilhelm Sjöberg, and Brent Yorgey. 2021. *Logical Foundations* (6.0 ed.). Software Foundations, Vol. 1. Electronic textbook, University of Pennsylvania. <https://softwarefoundations.cis.upenn.edu/lf-current.html>
- [28] J. Rushby, S. Owre, and N. Shankar. 1998. Subtypes for specifications: predicate subtyping in PVS. *IEEE Transactions on Software Engineering* 24, 9 (1998), 709–720. doi:10.1109/32.713327
- [29] Amr Sabry and Matthias Felleisen. 1992. Reasoning About Programs in Continuation-Passing Style. In *Proceedings of the 1992 ACM conference on LISP and functional programming* (San Francisco, California, USA) (LFP '92). Association for Computing Machinery, New York, NY, USA, 288–298. doi:10.1145/141471.141563
- [30] Taro Sekiyama, Atsushi Igarashi, and Michael Greenberg. 2017. Polymorphic Manifest Contracts, Revised and Resolved. *ACM Transactions on Programming Languages and Systems* 39, 1 (2017), 1–36. doi:10.1145/2994594
- [31] Taro Sekiyama, Yuki Nishida, and Atsushi Igarashi. 2015. Manifest Contracts for Datatypes. In *Proceedings of the 42nd Annual ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages* (Mumbai, India, 2015-01) (POPL '15). ACM, New York, NY, USA, 195–207. doi:10.1145/2676726.2676996
- [32] Matthieu Sozeau. 2007. Subset Coercions in Coq. In *Types for Proofs and Programs* (2007), Thorsten Altenkirch and Conor McBride (Eds.). Springer Berlin Heidelberg, 237–252. doi:10.1007/978-3-540-74464-1_16
- [33] Matthieu Sozeau and Cyprien Mangin. 2019. Equations Reloaded: High-level Dependently-Typed Functional Programming and Proving in Coq. In *Proceedings of the ACM on Programming Languages* (2019-07), Vol. 3. ACM, 1–29. doi:10.1145/3341690
- [34] Matthieu Sozeau and Nicolas Oury. 2008. *First-Class Type Classes*. Springer Berlin Heidelberg, 278–293. doi:10.1007/978-3-540-71067-7_23
- [35] Antal Spector-Zabusky, Joachim Breitner, Christine Rizkallah, and Stephanie Weirich. 2018. Total Haskell is Reasonable Coq. In *Proceedings of the 7th ACM SIGPLAN International Conference on Certified Programs and Proofs* (Los Angeles, CA, USA, 2018-01) (CPP '18). ACM, New York, NY, USA, 14–27. doi:10.1145/3167092
- [36] Martin Sulzmann, Manuel M. T. Chakravarty, Simon Peyton Jones, and Kevin Donnelly. 2007. System F with type equality coercions. In *Proceedings of the 2007 ACM SIGPLAN International Workshop on Types in Languages Design and Implementation* (Nice, Nice, France) (TLDI '07). Association for Computing Machinery, New York, NY, USA, 53–66. doi:10.1145/1190315.1190324
- [37] Nikhil Swamy, Cătălin Hrițcu, Chantal Keller, Aseem Rastogi, Antoine Delignat-Lavaud, Simon Forest, Karthikeyan Bhargavan, Cédric Fournet, Pierre-Yves Strub, Markulf Kohlweiss, Jean-Karim Zinzindohoue, and Santiago Zanella-Béguélin. 2016. Dependent types and multi-monadic effects in F*. In *Proceedings of the 43rd Annual ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages* (St. Petersburg, FL, USA) (POPL '16). Association for Computing Machinery, New York, NY, USA, 256–270. doi:10.1145/2837614.2837655
- [38] Sam Tobin-Hochstadt and Matthias Felleisen. 2010. Logical Types for Untyped Languages. In *Proceedings of the 15th ACM SIGPLAN international conference on Functional programming* (2010-09) (ICFP '10). ACM, 117–128.
- [39] Niki Vazou, Eric L. Seidel, Ranjit Jhala, Dimitrios Vytiniotis, and Simon Peyton-Jones. 2014. Refinement Types for Haskell. In *Proceedings of the 19th ACM SIGPLAN international conference on Functional programming* (Gothenburg, Sweden, 2014-08) (ICFP '14). ACM, New York, NY, USA, 269–282. doi:10.1145/2628136.2628161
- [40] Niki Vazou, Anish Tondwalkar, Vikraman Choudhury, Ryan G. Scott, Ryan R. Newton, Philip Wadler, and Ranjit Jhala. 2017. Refinement Reflection: Complete Verification with SMT. *Proceedings of the ACM on Programming Languages* 2, POPL, Article 53 (Dec. 2017), 31 pages. doi:10.1145/3158141

$$\begin{array}{c}
\frac{(x : R) \in \Gamma_c \quad c \in \{L, G\}}{\Gamma_G \uplus \Gamma_L \vdash_{\text{SIMPLE}} x : \text{unref}(R) \rightsquigarrow x/(\text{arity}(R), c)} \quad \frac{\Gamma \vdash_{\text{SIMPLE}} r_1 : A' \rightarrow A \rightsquigarrow r_1^e \quad \Gamma \vdash_{\text{SIMPLE}} r_2 : A' \rightsquigarrow r_2^e}{\Gamma \vdash_{\text{SIMPLE}} r_1 r_2 : A \rightsquigarrow r_1^e r_2^e} \\
\\
\frac{}{\Gamma \vdash_{\text{SIMPLE}} \ell : \text{unref}(\text{ty}(\ell)) \rightsquigarrow \ell} \quad \frac{\exists TC \in \text{dom}(\Gamma). (C : R) \in \Gamma(TC)}{\Gamma \vdash_{\text{SIMPLE}} C : \text{unref}(R) \rightsquigarrow C} \\
\\
\frac{\Gamma \vdash_{\text{SIMPLE}} r : \mathbf{Bool} \rightsquigarrow r^e}{\Gamma \vdash_{\text{SIMPLE}} \neg r : \mathbf{Bool} \rightsquigarrow \neg r^e} \quad \frac{\Gamma \vdash_{\text{SIMPLE}} r_1 : A_1 \rightsquigarrow r_1^e \quad \Gamma \vdash_{\text{SIMPLE}} r_2 : A_2 \rightsquigarrow r_2^e \quad A_1 = A_2}{\Gamma \vdash_{\text{SIMPLE}} r_1 = r_2 : \mathbf{Bool} \rightsquigarrow r_1^e = r_2^e} \\
\\
\frac{\text{unref}(\text{ty}(\odot)) = A_1 \rightarrow A_2 \rightarrow A \quad \Gamma \vdash_{\text{SIMPLE}} r_1 : A_1 \rightsquigarrow r_1^e \quad \Gamma \vdash_{\text{SIMPLE}} r_2 : A_2 \rightsquigarrow r_2^e}{\Gamma \vdash_{\text{SIMPLE}} r_1 \odot r_2 : A \rightsquigarrow r_1^e \odot r_2^e} \\
\\
\frac{\Gamma \vdash_{\text{SIMPLE}} r_1 : B \rightsquigarrow r_1^e \quad \Gamma \vdash_{\text{SIMPLE}} r_2 : \mathbf{Bool} \rightsquigarrow r_2^e}{\Gamma \vdash_{\text{SIMPLE}} r_1 ? r_2 : B \rightsquigarrow r_1^e ? r_2^e}
\end{array}$$

Fig. 4. Simple type system for refinements

A Full Type System for λ_r

In this section, we give all well-formedness and typing rules.

We have six kinds of judgement: well-formedness of types and of declarations, subtyping, type synthesis for refinements, type checking of expressions (including refinements), and a simple type system for type refinements. All except subtyping result have an elaboration.

To remember the locality of variables, we split environments Γ into a global environment Γ_G and a local one Γ_L . We still write Γ most of the time for $\Gamma_G \uplus \Gamma_L$. We write $\Gamma(x)$ to look up the variable x in Γ and $\text{dom}(\Gamma)$ to get the domain of Γ . We also write $\Gamma, x : R$ for $\Gamma_G \uplus \Gamma_L, x : R$ —that is, by default we add bindings to the local context.

$$\begin{array}{lll}
\textbf{Typing environment} & \Gamma & ::= \Gamma_G \uplus \Gamma_L \\
\textbf{Global environment} & \Gamma_G & ::= \emptyset \mid \Gamma_G, x : R \mid \Gamma_G, \langle TC, \overline{(C : R)} \rangle \\
\textbf{Local environment} & \Gamma_L & ::= \emptyset \mid \Gamma_L, x : R
\end{array}$$

A.1 Well-formedness and subtyping

Simple Type System for Type Refinements. We write A for a simple type, that is either a base type B or an arrow $A \rightarrow A$. We use the function $\text{unref}(R)$ to project a refined type into a type A . It is defined as $\text{unref}(\{x : B \mid r\}) \doteq B$ and $\text{unref}(x : R_x \rightarrow R) \doteq \text{unref}(R_x) \rightarrow \text{unref}(R)$.

The rules are in fig. 4. They are standard and similar to the synthesis rules. The only rule with a non-trivial elaboration is the one for variables.

Well-Formedness of Types. The substitution of a term in a type is defined as $\{x : B \mid r_x\} \{x \setminus r\} \doteq \{x : B \mid r_x\}$, $\{y : B \mid r_y\} \{x \setminus r\} \doteq \{y : B \mid r_y \{x \setminus r\}\}$, $(x : R_x \rightarrow R) \{x \setminus r\} \doteq x : R_x \rightarrow R$ and $(y : R_y \rightarrow R) \{x \setminus r\} \doteq y : R_y \{x \setminus r\} \rightarrow R \{x \setminus r\}$.

$$\frac{\text{if } B = TC, \text{ then } B \in \text{dom}(\Gamma) \quad \Gamma, x : \{B\} \vdash_{\text{SIMPLE}} r : \mathbf{Bool} \rightsquigarrow r^e}{\Gamma \vdash \{x : B \mid r\} \rightsquigarrow \{x : B \mid r^e\}} \text{E-TRF} \quad \frac{\Gamma \vdash R_x \rightsquigarrow R_x^e \quad \Gamma, x : R_x \vdash R \rightsquigarrow R^e}{\Gamma \vdash x : R_x \rightarrow R \rightsquigarrow x : R_x^e \rightarrow R^e \{x \setminus \mathbf{proj}(x)\}} \text{E-TFUN}$$

Well-Formedness of Declarations. Well-formedness of declarations takes as input a typing environment with no local variables and a sequence of declarations.

$$\frac{\frac{\Gamma_G, \langle TC, (C_k : R_k^e)_{k < i} \rangle \uplus \emptyset \vdash R_i \rightsquigarrow R_i^e \quad R_i^e = (x_j : \{_ : _ \mid _ \})_{j \leq m_i} \rightarrow R_i^e}{\Gamma_G, \langle TC, (C_i : R_i^e)_{i \leq n} \rangle \uplus \emptyset \vdash \overline{D} \rightsquigarrow \overline{D}^e} \quad \frac{\Gamma_G, \overline{C}_i \text{ fresh for } \Gamma_G}{\Gamma_G \uplus \emptyset \vdash \mathbf{data} \ TC := (C_i :: R_i)_{i \leq n}; \overline{D} \rightsquigarrow \mathbf{data} \ TC := (C_i :: R_i^e)_{i \leq n}; \overline{D}^e} \text{WF-DTC}$$

Rule WF-DTC requires a fresh type constructor TC and fresh data constructors C_i with well-formed, first-order types returning a refinement of TC . First-orderness is ensured because every argument to a constructor refines a base type. The rule recursively checks the well-formedness of the subsequent declarations.

$$\frac{\frac{\Gamma_G \uplus \emptyset \vdash R_f \rightsquigarrow R_f^e \quad R_f^e = (x_i : R_i)_{i \leq n} \rightarrow R}{\Gamma_G, f : R_f^e \uplus (x_i : R_i)_{i \leq n} \vdash e \Leftarrow R \rightsquigarrow e^e} \quad \frac{\Gamma_G, f : R_f^e \uplus \emptyset \vdash \overline{D} \rightsquigarrow \overline{D}^e}{\Gamma_G \uplus \emptyset \vdash (\mathbf{reflect})^? \ \mathbf{def} \ f :: R_f := e; \overline{D} \rightsquigarrow (\mathbf{reflect})^? \ \mathbf{def} \ f :: R_f^e := e^e; \overline{D}^e} \text{WF-DDEF}$$

Rule WF-DDEF checks the body of f using its “maximal” function type—the return type R must not be a function type. That is, we forbid point-free definitions and require complete η -expansion.

Subtyping. Subtyping has two rules: one for base types and one for function types. The left rule only checks that the base types are equal.

$$\frac{}{\Gamma \vdash \{x_s : B \mid r_s\} <: \{x_t : B \mid r_t\}} \text{S-BASE} \quad \frac{\Gamma \vdash R_{x_t} <: R_{x_s} \quad \Gamma, x_t : R_{x_t} \vdash R_s \{x_s \setminus x_t\} <: R_t}{\Gamma \vdash x_s : R_{x_s} \rightarrow R_s <: x_t : R_{x_t} \rightarrow R_t} \text{S-ARR}$$

A.2 Typing Expressions

We have a bidirectional type system where synthesis is done exclusively on refinements, and checking is done on structural expressions, and always ends with a subtyping rule when done on refinements. A synthesis rule $\Gamma \vdash r \Rightarrow R \rightsquigarrow r^e$ inputs Γ and r , synthesizes the type R and outputs e^e . A checking rule $\Gamma \vdash e \Leftarrow R \rightsquigarrow e^e$ inputs Γ , e and R and checks that e has the refinement type R , and outputs e^e .

The type synthesis rules for refinements are given in fig. 5. The types of literals and operators are given by a function $\text{ty}(\cdot)$ that generates a singleton type (or *selfified* type [26]). Typing equality is similar, but works for any base type.

The type checking rules are given in fig. 6.

Rule C-LET has two subtleties. The first is that, if R_x is an arrow type, we check e_1 in the context extended by its parameters x_1 through x_n . The second is that we check well-formedness of the type R : this is to ensure that this type does not depend on the defined variable x .

In the rules for if and case, we use occurrence typing [38]: each branch is checked with an additional information on the value of the condition or match. The typing environment in C-CASE is carefully strengthened with all the information gathered from the refinement types of the data constructors and the matched variable. All branches must be present, but some can be **unreachable**.

B Appendix for the Translations

Typeclass for projection. Our notation $\llbracket \cdot \rrbracket$ in Rocq is generic over subset types and Packs. We define it generically in the implementation as well, as part of a typeclass. The operator $\llbracket \cdot \rrbracket$ is an alias for the function proj of the typeclass. This function must respect a “proof irrelevance” property po ensuring equality modulo potential refinements.

$$\begin{array}{c}
\frac{(x : R) \in \Gamma_L \quad \text{arity}(R) = 0}{\Gamma_G \uplus \Gamma_L \vdash x \Rightarrow R \rightsquigarrow \mathbf{inj}(x/(0,L), R)} \text{S-VARL} \\
\frac{(x : R) \in \Gamma_{c'} \quad c' \in L, G \quad \text{if } c' = L \text{ then } \text{arity}(R) > 0 \\ \text{if } \Gamma_G = \Gamma'_G, (x : R) \text{ then } c = Y \text{ else } c = c'}{\Gamma_G \uplus \Gamma_L \vdash x \Rightarrow R \rightsquigarrow x/(\text{arity}(R), c)} \text{S-VAR} \\
\frac{}{\Gamma \vdash \ell \Rightarrow \text{ty}(\ell) \rightsquigarrow \mathbf{inj}(\ell, \text{ty}(\ell))} \text{S-LIT} \quad \frac{\exists TC \in \text{dom}(\Gamma). (C : R) \in \Gamma(TC)}{\Gamma \vdash C \Rightarrow R \rightsquigarrow C} \text{S-DATA} \\
\frac{\text{ty}(\neg) = x : R_x \rightarrow R \quad \Gamma \vdash r \Leftarrow R_x \rightsquigarrow r^e}{\Gamma \vdash \neg r \Rightarrow R\{x \setminus r^e\} \rightsquigarrow \neg r^e} \text{S-NEG} \quad \frac{\text{ty}(\odot) = x_1 : R_1 \rightarrow x_2 : R_2 \rightarrow R \quad \Gamma \vdash r_1 \Leftarrow R_1 \rightsquigarrow r_1^e \quad \Gamma \vdash r_2 \Leftarrow R_2 \rightsquigarrow r_2^e}{\Gamma \vdash r_1 \odot r_2 \Rightarrow R\{x_1 \setminus r_1^e\} \{x_2 \setminus r_2^e\} \rightsquigarrow r_1^e \odot r_2^e} \text{S-BIN} \\
\frac{\Gamma \vdash r_1 \Rightarrow \{x_1 : B_1 \mid r_1\} \rightsquigarrow r_1^e \quad \Gamma \vdash r_2 \Rightarrow \{x_2 : B_2 \mid r_2\} \rightsquigarrow r_2^e \quad B_1 = B_2}{\Gamma \vdash r_1 = r_2 \Rightarrow \{x : \mathbf{Bool} \mid x = (\mathbf{proj}(r_1^e) = \mathbf{proj}(r_2^e))\} \rightsquigarrow r_1^e = r_2^e} \text{S-EQ} \\
\frac{\Gamma \vdash r_1 \Rightarrow x : R_x \rightarrow R \rightsquigarrow r_1^e \quad \Gamma \vdash r_2 \Leftarrow R_x \rightsquigarrow r_2^e}{\Gamma \vdash r_1 r_2 \Rightarrow R\{x \setminus r_2^e\} \rightsquigarrow r_1^e r_2^e} \text{S-APP} \\
\frac{\Gamma \vdash r_h \Rightarrow \{x : () \mid r_p\} \rightsquigarrow r_h^e \quad \Gamma, x : \{x : () \mid r_p\} \vdash r \Rightarrow R \rightsquigarrow r^e}{\Gamma \vdash r ? r_h \Rightarrow R \rightsquigarrow r^e ? (r_h^e \text{ proves } r_p)} \text{S-HINT}
\end{array}$$

Fig. 5. Type synthesis rules for refinements

$$\begin{array}{c}
\frac{\Gamma \vdash R_x \rightsquigarrow R_x^e \quad R_x^e = (x_i : R_i)_{i \leq n} \rightarrow R' \quad \Gamma, (x_i : R_i)_{i \leq n} \vdash e_1 \Leftarrow R' \rightsquigarrow e_1^e \quad \Gamma, x : R_x^e \vdash e_2 \Leftarrow R \rightsquigarrow e_2^e \quad \Gamma \vdash R \rightsquigarrow R^e}{\Gamma \vdash \mathbf{let } x :: R_x := e_1 \mathbf{ in } e_2 \Leftarrow R \rightsquigarrow \mathbf{let } x :: R_x^e := e_1^e \mathbf{ in } e_2^e} \text{C-LET} \\
\frac{\Gamma \vdash r \Rightarrow \{v : \mathbf{Bool} \mid r'\} \rightsquigarrow r^e \quad z \text{ fresh} \quad \Gamma, z : \{v : () \mid \mathbf{proj}(r^e) \wedge r'\} \vdash e_1 \Leftarrow R \rightsquigarrow e_1^e \quad \Gamma, z : \{v : () \mid \neg \mathbf{proj}(r^e) \wedge r'\} \vdash e_2 \Leftarrow R \rightsquigarrow e_2^e}{\Gamma \vdash \mathbf{if } r \text{ then } e_1 \mathbf{ else } e_2 \Leftarrow R \rightsquigarrow \mathbf{if } r^e \text{ then } e_1^e \mathbf{ else } e_2^e} \text{C-IF} \\
\frac{\Gamma \vdash r \Rightarrow \{v : TC \mid r_v\} \rightsquigarrow r^e \quad \langle TC, (C : (y_i^j : R_i^j)_{j \leq m_i} \rightarrow \{v : TC \mid r_i\})_{i \leq n} \rangle \in \Gamma \quad I \subseteq \{1, \dots, n\} \\ \Gamma_i \doteq \Gamma, (y_i^j : R_i^j)_{j \leq m_i}, z : \{v : TC \mid \mathbf{proj}(R) = C_i y_i^1 \dots y_i^{m_i} \wedge r_v \wedge r_i\} \quad z \text{ fresh} \\ \forall i \leq n \text{ s.t. } e_i \neq \mathbf{unreachable}. (\Gamma_i \vdash e_i \Leftarrow R \rightsquigarrow e_i^e)}{\Gamma \vdash \mathbf{case } r \mathbf{ of } (C_i y_i^1 \dots y_i^{m_i} \mapsto e_i)_{i \leq n} \Leftarrow R \rightsquigarrow \mathbf{case } r^e \mathbf{ of } (C_i y_i^1 \dots y_i^{m_i} \mapsto e_i^e)_{i \leq n}} \text{C-CASE} \\
\frac{\Gamma \vdash r \Rightarrow R_s \rightsquigarrow r^e \quad \Gamma \vdash R_s <: R_t}{\Gamma \vdash r \Leftarrow R_t \rightsquigarrow \mathbf{sub}(r^e, R_s, R_t)} \text{C-SYN}
\end{array}$$

Fig. 6. Type checking rules for expressions

Class Projection $\{R \cup : \text{Type}\} := \{\text{proj} : R \rightarrow U; \text{po} : \forall (x \ y : R), \text{proj } x = \text{proj } y \rightarrow x = y\}.$

Global Notation $\llbracket x \rrbracket := (\text{proj } x).$

For a subset type, `proj` is defined simply as Rocq's `proj1_sig`. For a `Pack`, we define the projection from a `Pack` to an `uPack` simply as $\text{projPack}(x) \doteq \{|\text{rel_u} := x.\text{rel}; \text{funct_u} := x.\text{pack}.\text{funct}|\}$. The parameters `R` and `U` are implicit, as indicated by the curly brackets. Rocq can always infer the correct instance automatically, since we define the projection operator only on subset types and `Packs`.

Call hypotheses. We give a thorough definition of the function $\text{Hyp}(r)$ used in § 3.4 to close the term $\text{RtoU}(r)$ with the hypotheses originating from the function and operator calls.

Definition B.1 (Call hypotheses). Given a refinement r , and assuming the map RV , $\text{Hyp}(r)$ is defined as:

$$\begin{aligned} \text{Hyp}(r) \doteq & \{ \forall z, f_{\text{rel}} \text{ RtoU}(r_1) \dots \text{RtoU}(r_n) z \quad (* f r_1 \dots r_n \mapsto z \in \text{RV} *) \\ & \mid f_{/(n,G)} r_1 \dots r_n \text{ subterm of } r \wedge n \geq 0 \} \\ \cup & \{ \forall z, (\text{getUPackRel } f) \text{ RtoU}(r_1) \dots \text{RtoU}(r_n) z \quad (* f r_1 \dots r_n \mapsto z \in \text{RV} *) \\ & \mid f_{/(n,L)} r_1 \dots r_n \text{ subterm of } r \wedge n > 0 \} \\ \cup & \{ \forall z, (\text{getPackRel } f) \text{ RtoU}(r_1) \dots \text{RtoU}(r_n) z \quad (* \text{proj}(f) r_1 \dots r_n \mapsto z \in \text{RV} *) \\ & \mid \text{proj}(f_{/(n,L)}) r_1 \dots r_n \text{ subterm of } r \wedge n > 0 \} \\ \cup & \{ \forall z, \odot_{\text{rel}} \text{ RtoU}(r_1) \text{ RtoU}(r_2) z \mid r_1 \odot r_2 \text{ subterm of } r \} (* r_1 \odot r_2 \mapsto z \in \text{RV} *) \\ \cup & \{ \forall z, \neg_{\text{rel}} \text{ RtoU}(r_0) z \mid \neg r_0 \text{ subterm of } r \} (* \neg r_0 \mapsto z \in \text{RV} *) \end{aligned}$$

Compared to the textual description from § 3.4, here we use `getUPackRel f` and `getPackRel f` to obtain the relation from a `Pack`, going through the conversion from the representation inside the `Pack` to the one we expect (see appendix D).

Retrieving the function paths. We now describe how we compute the paths of a reflected function, necessary to build its unrefined graph relation.

Definition B.2. Consider a reflected function f , not containing λ -abstractions or partial function applications, defined as **(reflect)**[?] **def** $f :: (x_i : R_i)_{i \leq n} \rightarrow R := e$. The paths of f are given by an auxiliary function $\mathcal{P}$.

$$\text{paths}(e; \bar{x}) \doteq \{ (\text{codom}(\sigma_{xs}), \sigma_p, e_p) \mid (\sigma_{xs}, \sigma_p, e_p) \in \mathcal{P}(\overline{(x, x)}, \emptyset, e) \}$$

To simplify and without loss of generality, we consider in this section the literals **True** and **False** as data constructors, and encode conditionals as case expressions. The function $\mathcal{P}$ takes as input a triple corresponding to the current branch and returns a set of branches where matches on the arguments are put first, and nested pattern matchings are merged into one. The arguments are thus: (1) a map from variables to simple terms, the patterns for the parameters, (2) a map from additional matched refinements to their pattern, and (3) the expression e to turn into branches.

Definition B.3. Given a map from variables to simple terms σ_{xs} , a map from simple terms to simple terms σ_p and an expression e , $\mathcal{P}(\sigma_{xs}, \sigma_p, e)$ is defined as:

$$\begin{aligned} \mathcal{P}(\sigma_{xs}, \sigma_p, r) & \doteq \{ (\sigma_{xs}, \sigma_p, r) \} \\ \mathcal{P}(\sigma_{xs}, \sigma_p, \text{unreachable}) & \doteq \emptyset \\ \mathcal{P}(\sigma_{xs}, \sigma_p, \text{let } x :: R := e_1 \text{ in } e_2) & \doteq \bigcup_{(\sigma'_{xs}, \sigma'_p, r) \in \mathcal{P}(\sigma_{xs}, \sigma_p, e_1)} \mathcal{P}(\sigma'_{xs}, \sigma'_p, e_2 \{x \setminus r\}) \\ \mathcal{P}(\sigma_{xs}, \sigma_p, \text{case } r \text{ of } (C_i \overline{y_i} \mapsto e_i)_{i \leq n}) & \\ \doteq & \begin{cases} \mathcal{P}(\sigma_{xs}, \sigma_p, e_j \{y_i \setminus r_i\}_{i \leq m}) & \text{if } \exists j \leq n. r = C_j r_1 \dots r_m \text{ or } \exists (r, C_j r_1 \dots r_m) \in \sigma_{xs} \cup \sigma_p \\ \bigcup_{i \leq n} \mathcal{P}(\sigma_{xs} \{x \setminus C_i \overline{y_i}\}, \sigma_p \{x \setminus C_i \overline{y_i}\}, e_i \{x \setminus C_i \overline{y_i}\}) & \text{if } r = x \\ \bigcup_{i \leq n} \mathcal{P}(\sigma_{xs}, \sigma_p, (r, C_i \overline{y_i}), e_i) & \text{otherwise} \end{cases} \end{aligned}$$

Where we write $\sigma_{xs} \{x \setminus C_i \overline{y_i}\}$ to substitute in the codomain of σ_{xs} and $\sigma_p \{x \setminus C_i \overline{y_i}\}$ to substitute either in the domain or codomain of σ_p .

$$\begin{array}{l}
\overline{D} \vdash r \rightarrow_{\text{er}} \delta(r) \quad \text{if } r \in \{\neg \ell, \ell_1 \odot \ell_2, v_1 = v_2\} \\
\overline{D} \vdash \mathbf{inj}(r, R) \rightarrow_{\text{er}} r' \\
\overline{D} \vdash \mathbf{case } C_k \overline{v_k} \mathbf{ of } (C_i \overline{x_i} \mapsto e_i)_{i \leq n} \rightarrow_{\text{er}} \overline{e_k \{x_k \setminus v_k\}} \\
\overline{D} \vdash \mathbf{let } x :: R := v \mathbf{ in } e \rightarrow_{\text{er}} e \{x \setminus v\} \\
\overline{D} \vdash f v_1 \dots v_n \rightarrow_{\text{er}} e \{x_i \setminus v_i\}_{i \leq n} \quad \text{if } \mathbf{reflect def } f :: (x_i : R_i)_{i \leq n} \rightarrow R := e \in \overline{D}
\end{array}$$

Where δ is a function returning the correct value for primitive operations.

Fig. 7. Reduction relation

There are two base cases: either we find a branch with a refinement at the end, or we find an **unreachable** branch, that we delete. This way, we ensure that no **unreachable** appears in the function paths. Let-bindings are substituted away: this is possible since all branches lead to a refinement, that can be substituted anywhere, unlike expressions. This would not be possible with λ -abstractions or partial applications.

There are three equations for pattern matching. A match on an application (last case) creates a new pattern in the map σ_p . A matched variable can be either a parameter of the function or a variable introduced by pattern matching. We substitute this variable by its pattern in the image of σ_{xs} and everywhere in the second case. The first case accounts for a match on an already matched variable or application, in which case there is only one valid pattern.

C Proofs for § 4

C.1 Preservation of Evaluation Semantics of Expressions

In this subsection, we look at the operational semantics of reflected functions. We define a reduction semantics for expressions with no higher-order let-bindings (that is, bound λ -abstractions). Like in appendix B, we encode conditionals as case expressions without loss of generality. The goal is to show that the operational semantics of the expression is preserved by the translation to the graph relation.

Definition C.1 (Small-steps reduction). We equip λ_r with a small-steps call-by-value reduction. The base reduction rules are given in fig. 7. They are applied under an environment of declarations $\overline{D}$ from which we retrieve the definition of free variables. Reduction is closed under all contexts. We write $\rightarrow_{\text{er}}^*$ for the recursive and transitive closure of $\rightarrow_{\text{er}}$.

Values are defined as $v ::= x \mid \ell \mid C \overline{v}$, where x is not a constant of $\overline{D}$. Normal forms of the reduction of a term closed in the declaration environment are closed values.

We assume that the reduction terminates, as Liquid Haskell checks termination of programs. Therefore, we can prove confluence simply by proving local confluence.

LEMMA C.2 (CONFLUENCE). *The reduction on λ_r is confluent.*

PROOF. We show local confluence and conclude by termination of the reduction. There are three kinds of critical pairs. The first is when $\overline{D} \vdash r ? (r_h \text{ proves } r_p) \rightarrow_{\text{er}} r$ and either r_h or r_p is reduced (as r'_h or r'_p). We reduce $\overline{D} \vdash r ? (r'_h \text{ proves } r'_p) \rightarrow_{\text{er}} r$. The second kind is trivial and comes from the reduction of two subterms that do not interact, as $r_1 \odot r_2$ that can reduce both r_1 and r_2 .

The third kind concerns pattern matching and let, both cases are similar. We can reduce $\mathbf{let } x :: R := v \mathbf{ in } e$ both as $e \{x \setminus v\}$ and $\mathbf{let } x :: R := v \mathbf{ in } e'$. To close the diagram with $e' \{x \setminus v\}$, we show the following property: given an expression e such that $\overline{D} \vdash e \rightarrow_{\text{er}} e'$ and a substitution

of variables to values σ , we have $\overline{D} \vdash e\sigma \rightarrow_{\text{er}} e'\sigma$. This property is easily proved by induction on $\overline{D} \vdash e \rightarrow_{\text{er}} e'$. $\square$

The graph relation of a function is created by reorganizing the structure of the term into the function paths. The function $\mathcal{P}$ from definition B.3 only commutes pattern matchings, merges them and eliminate superfluous matchings. Since the reduction is confluent, these operations are harmless for the final result: we could define a function to convert the set of function paths back into an expression, and both the original function and that expression would reach the same normal form. We formalize this below with an additional detail: to be closer to the representation of the graph relation in Rocq, we build an λ_r expression from the function paths, but where we also extract function and operator calls into let-bindings.

Definition C.3. Consider a set $P \doteq \{(\sigma_{xs}^i, \sigma_p^i, r_i)\}_{i \leq n}$ of paths obtained by $\mathcal{P}$. We transform this set back into an expression with $\text{caseify}(P) \doteq \text{caseify_aux}(\{\sigma_i, r_i\}_{i \leq n}, \emptyset)$, where σ_i is the concatenation of σ_{xs}^i and σ_p^i where we destructured back the pattern: for instance, a tuple $(r, C (C_1 x_1) x_2 C_2)$ is replaced by the sequence $(r, C y x_2 z), (y, C_1 x_1), (z, C_2)$, for fresh variables y and z .

To mimic the construction of the graph relation (definition 3.5), we consider the map RV from calls to free variables and we assume a function $\text{repl}(r)$ that replaces the applications in r according to RV. The second argument of caseify_aux is a set $H \subset \text{RV}$ similar to the one used in definition 3.5.

$$\begin{aligned} \text{caseify_aux}(\{(\emptyset, r)\}, H) &\doteq \text{let } (z := \text{repl}(r_z))_{(z, r_z) \in H'} \text{ in } \text{repl}(r) \\ \text{caseify_aux}(\{((r, r_i) : \sigma_i^j, r_i^j)\}_{j \leq m_i, i \leq n}, H) &\doteq \text{let } (z := \text{repl}(r_z))_{(z, r_z) \in H'} \text{ in} \\ &\quad \text{case } r' \text{ of } (r_i \mapsto \text{caseify_aux}(\{(\sigma_i^j, r_i^j)\}_{j \leq m_i}, H \cup H'))_{i \leq n} \end{aligned}$$

Where in the second case $r' \doteq \text{repl}(r)$ if r is an application and $r' \doteq x$ if $r = x$, and in both cases $H' \doteq \{(z, r_z) \in \text{RV} \setminus H \mid r_z \text{ is a subterm of } r\}$.

LEMMA C.4. Let $\overline{D} \doteq \overline{D}'$; **reflect def** $f :: (x_i : R_i)_{i \leq n} \rightarrow R := e \in \overline{D}$ be a well-formed and annotated sequence of reflected declarations. Let $v_1, \dots, v_n$ be closed under $\overline{D}$. If $\overline{D} \vdash e\{x_i \setminus v_i\}_{i \leq n} \rightarrow_{\text{er}}^* v$, then $\overline{D} \vdash \text{caseify}(\text{paths}(e; (x_i)_{i \leq n}))\{x_i \setminus v_i\}_{i \leq n} \rightarrow_{\text{er}}^* v$.

PROOF. By confluence of the reduction, $\mathcal{P}$ preserves the result of evaluation. Confluence is what allows us to extract applications inside let-bindings. $\square$

THEOREM 4.1. Let $\overline{D}$ be a well-formed and annotated sequence of declarations, with all functions reflected. Let **reflect def** $f :: (x_i : R_i)_{i \leq n} \rightarrow R := e_f \in \overline{D}$, $v_1, \dots, v_n, v$ be values (variables, literals and constructors) such that $\overline{D} \vdash f v_1 \dots v_n \rightarrow_{\text{er}}^* v$. Then, in Rocq and under $\text{DtoR}(\overline{D})$:

- (i) $f_rel \text{ RtoU}(v_1) \dots \text{RtoU}(v_n) \text{ RtoU}(v)$ holds and (ii) $\llbracket \text{RtoR}(f v_1 \dots v_n) \rrbracket [(x_i)_{i \leq n}] = \text{RtoU}(v)$.

PROOF. The relation between the unrefined graph relation and the refined translation of f (item (ii)) is shown directly in Rocq, in the lemmas we described in § 3.3.2. Let us consider item (i).

First, notice that the property holds for operators by definition of their relation built with Rocq's builtin operators. We have $\overline{D} \vdash f v_1 \dots v_n \rightarrow_{\text{er}}^* e_f\{x_i \setminus v_i\}_{i \leq n}$. Thanks to the previous lemmas, we can go from the initial reduction on e_f to a reduction on $\text{caseify}(\text{paths}(e_f; (x_i)_{i \leq n}))$.

Since $f v_1 \dots v_n$ reduces to a value, none of the intermediate computations is stuck on a pattern matching. Therefore, there is a path $p = (\sigma_{xs}, \sigma_r, r_p) \in \text{paths}(e_f; (x_i)_{i \leq n})$ corresponding to the reduction.

The proof is by induction on the reduction. The structure of caseify models that of the constructor f^p . From the matching of the arguments, we obtain an instantiation of the free variables of the patterns in σ_{xs} , with which we instantiate the first variables of f^p .

Then, let us consider the matchings on additional applications. The hypotheses introduced in f^p are in one-to-one correspondance with the let-bindings introduced in `caseify`. These hypotheses are of the shape $\forall z, g_0 \in_1 \dots \in_m \mathbb{E}$, where g_0 is `g_rel` or `getUPackRel g` or `getPackRel g` and where $\in_1$ to $\in_m$ and $\mathbb{E}$ are Rocq expressions obtained by translation. The encoding of a relation inside a `Pack` preserves its operational semantics, so that in all cases we can apply the induction hypothesis, since this specific application of g appears in the translation. This gives us an instantiation from the fresh variables of the hypotheses to the result of the applications. The evaluation of the pattern matching gives us an instantiation of the free variables of the pattern.

Now, let σ be the substitution obtained by the series of pattern matching and let-bindings. Given z in $\text{dom}(\sigma)$, there are two possibilities: either $\sigma(z) = v_z$ for $\overline{D} \vdash g \overline{v_g} \rightarrow_{\text{er}}^* v_z$, where g was bound in a let-binding, and z comes from RV, or $\sigma(z)$ is an arbitrary value, when z is a parameter or introduced with a pattern. For the return type of the constructor f^p , we instantiate the first n arguments of `f_rel` correctly thanks to the previous instantiations. For the $n + 1$ th argument (the result), we use the fact that for every refinement r , if there is v such that $\overline{D} \vdash r \rightarrow_{\text{er}} v$, then $\text{RtoU}(r)\sigma$ is equal modulo conversion to v in Rocq. This proof is direct by induction on $\overline{D} \vdash r \rightarrow_{\text{er}}^* r_2$. $\square$

C.2 Preservation of Models of Types

C.2.1 Simply-typed λ -calculus. We use terms of the simply-typed λ -calculus as a common interpretation for λ_r and Rocq.

Definition C.5 (Calculus $\lambda_{\rightarrow}$). The grammar of $\lambda_{\rightarrow}$ is given in fig. 8. Datatype declarations will be assumed in the typing context, so we don't have a way of defining them in the calculus. Operators and literals are taken to be the same as in λ_r .

Typing contexts	$\Gamma ::= \emptyset \mid \Gamma, x : A \mid \Gamma, \langle TC, \overline{(C : A)} \rangle$
Types	$A ::= () \mid \mathbf{Bool} \mid \mathbf{Int} \mid TC \mid A \rightarrow A$
Expressions	$e ::= x \mid v \mid C \mid ee \mid e \odot e \mid e = e \mid \neg e \mid \mathbf{case } e \text{ of } \overline{(C \overline{y} \mapsto e)}$
Values	$v ::= \ell \mid C \overline{v} \mid \lambda x. e$
Primitives	$c ::= () \mid \mathbf{True} \mid \mathbf{False} \mid 0, 1, -1 \dots$

Fig. 8. Grammar of the simply-typed λ -calculus

We take **Int** to be bounded and finitely represented. We assume the bounds for **Int** to be the same as in Liquid Haskell, with an undefined behavior for overflows. The reduction semantics of the calculus is standard, and called $\rightarrow_{\lambda}$. We reduce under all contexts except abstractions.

We define two translations, for λ_r and for the image of the translation of types, to the λ -calculus. In particular, we don't give a translation for arbitrary expressions of λ_r , but only refinements. The image of the translation of refinement types of λ_r is formally defined in fig. 9. In this section, we use the simplified definition of pack without encoding (see appendix D), as both versions have the same semantics.

Definition C.6 (Translation from λ_r types to the λ -calculus). We define a translation $[\cdot]_{H\lambda}$ from λ_r to $\lambda_{\rightarrow}$, over:

- Base types as the identity and refinement types as $\{x:B \mid r\}_{H\lambda} \doteq B$ and $[x:R_x \rightarrow R]_{H\lambda} \doteq [R_x]_{H\lambda} \rightarrow [R]_{H\lambda}$.
- Refinements as $[x/(_)]_{H\lambda} \doteq x$, $[r?(r_h \text{ proves } r_p)]_{H\lambda}$, $[\mathbf{inj}(r, _)]_{H\lambda} \doteq r$, $[\mathbf{sub}(r, _)]_{H\lambda} \doteq r$, $[\mathbf{proj}(r)]_{H\lambda} \doteq r$ and the identity for the rest.

Base types	$\mathbb{U} ::= \text{Unit} \mid \text{bool} \mid \mathbb{Z} \mid TC_u$
Subset types	$\$::= \{x:\mathbb{U} \mid Q\} \mid \forall(x:\$, \$) \mid \text{Pack}(\overline{\forall(x:\$)}, \$)$
Refinements	$\mathbb{P} ::= x \mid \lfloor x \rfloor \mid \ell \mid C_u \overline{\mathbb{P}} \mid \mathbb{P} ==? \mathbb{P}$
Literals	$\ell ::= \mathbf{tt} \mid \mathbf{true} \mid \mathbf{false} \mid 0, 1, -1, 2 \dots$
Propositions	$Q ::= \mathbf{is_true}(\mathbb{P}) \mid Q \wedge Q \mid \text{wf}_{TC} x \mid \mathbb{P} == \mathbb{P} \mid \forall z, (f_rel \mid f.(\text{rel}) \mid f.(\text{rel_u})) \overline{\mathbb{P}} z \rightarrow Q$

Fig. 9. Image of the translation of types in Rocq.

- Environments as $\lfloor \emptyset \rfloor_{H\lambda} \doteq \emptyset$, $\lfloor \Gamma, x : A \rfloor_{H\lambda} \doteq \lfloor \Gamma \rfloor_{H\lambda}, x : \lfloor A \rfloor_{H\lambda}$ and $\lfloor \Gamma, \langle TC, (C_i : A_i)_{i \in I} \rangle \rfloor_{H\lambda} \doteq \lfloor \Gamma \rfloor_{H\lambda}, \langle TC, (C_i : \lfloor A_i \rfloor_{H\lambda})_{i \in I} \rangle$.

Definition C.7 (Translation from the target Rocq grammar to the λ -calculus). We define a translation $\lfloor \cdot \rfloor_{Q\lambda}$ from the Rocq target to $\lambda_{\rightarrow}$ over:

- Base types: $\lfloor \text{Unit} \rfloor_{Q\lambda} \doteq \mathbf{()}$, $\lfloor \text{bool} \rfloor_{Q\lambda} \doteq \mathbf{Bool}$, $\lfloor \mathbb{Z} \rfloor_{Q\lambda} \doteq \mathbf{Int}$ and $\lfloor TC_u \rfloor_{Q\lambda} \doteq TC$.
- Refinement types: $\lfloor \{x:\mathbb{U} \mid \mathbb{P}\} \rfloor_{Q\lambda} \doteq \lfloor \mathbb{U} \rfloor_{Q\lambda}, \lfloor \forall(x:\$, \$) \rfloor_{Q\lambda} \doteq \lfloor \$ \rfloor_{Q\lambda} \rightarrow \lfloor \$ \rfloor_{Q\lambda}$ and $\lfloor \text{Pack}(\overline{\forall(x:\$)}, \$) \rfloor_{Q\lambda} \doteq \lfloor \$ \rfloor_{Q\lambda} \rightarrow \dots \rightarrow \lfloor \$ \rfloor_{Q\lambda} \rightarrow \lfloor \$ \rfloor_{Q\lambda}$.
- Literals: $\lfloor \mathbf{tt} \rfloor_{Q\lambda} \doteq \mathbf{()}$, $\lfloor \mathbf{true} \rfloor_{Q\lambda} \doteq \mathbf{True}$, $\lfloor \mathbf{false} \rfloor_{Q\lambda} \doteq \mathbf{False}$ and $\lfloor \ell \rfloor_{Q\lambda} \doteq \ell$ if ℓ is an integer.
- Refinements: $\lfloor x \rfloor_{Q\lambda} \doteq x$, $\lfloor \lfloor x \rfloor \rfloor_{Q\lambda} \doteq x$, $\lfloor C_u \overline{\mathbb{P}} \rfloor_{Q\lambda} \doteq C \overline{\lfloor \mathbb{P} \rfloor_{Q\lambda}}$ and $\lfloor \mathbb{P}_1 ==? \mathbb{P}_2 \rfloor_{Q\lambda} \doteq \lfloor \mathbb{P}_1 \rfloor_{Q\lambda} = \lfloor \mathbb{P}_2 \rfloor_{Q\lambda}$.
- Propositions: $\lfloor \mathbf{is_true}(\mathbb{P}) \rfloor_{Q\lambda} \doteq \lfloor \mathbb{P} \rfloor_{Q\lambda}$, $\lfloor Q_1 \wedge Q_2 \rfloor_{Q\lambda} \doteq \lfloor Q_1 \rfloor_{Q\lambda} \wedge \lfloor Q_2 \rfloor_{Q\lambda}$ (where $\wedge$ is the builtin boolean operator in $\lambda_{\rightarrow}$), $\lfloor Q_1 == Q_2 \rfloor_{Q\lambda} \doteq \lfloor Q_1 \rfloor_{Q\lambda} = \lfloor Q_2 \rfloor_{Q\lambda}$, $\lfloor \forall z, (f_rel/f.(\text{rel})/f.(\text{rel_u})) \overline{\mathbb{P}} z \rightarrow Q \rfloor_{Q\lambda} \doteq \lfloor Q \rfloor_{Q\lambda} \{z \mid f \overline{\lfloor \mathbb{P} \rfloor_{Q\lambda}}\}$, $\lfloor \text{wf}_{TC} x \rfloor_{Q\lambda} \doteq \mathbf{case} \ x \ \mathbf{of} \ (C_i \ x_1 \dots x_{m_i} \mapsto \lfloor Q_i \rfloor_{Q\lambda})_{i \in I} \ \text{if} \ \text{wf}_{TC} \ x \ \text{is defined as} \ \text{match} \ x \ \text{with} \ (C_i \ x_1 \dots x_{m_i} \mapsto Q_i)_{i \in I} \ \text{end}$.

We define two interpretations of types of λ_r and of the Rocq target respectively to sets of *closed* values of $\lambda_{\rightarrow}$. The interpretations are parametrized by an assignment ρ from variables to values of $\lambda_{\rightarrow}$ and by a λ_r environment Γ of datatypes only. We use the notation $\rho\{x \leftarrow v\}$ to indicate the update of the value of x to v in ρ .

Definition C.8 (Interpretation of λ_r types).

$$\begin{aligned}
\llbracket B \rrbracket_{\rho}^{\Gamma} &\doteq \{\ell \mid \emptyset \vdash_{\mathcal{L}C} \ell : B\} \quad B \text{ is a builtin type} \\
\llbracket TC \rrbracket_{\rho}^{\Gamma} &\doteq \{v = C \ v_1 \dots v_n \mid \exists (R_i)_{i \leq n}, x, r. \Gamma(TC)(C) = (x_i : R_i)_{i \leq n} \rightarrow \{x:TC \mid r\} \\
&\quad \wedge (\forall i \in [1..n]. v_i \in \llbracket R_i \rrbracket_{\rho\{x_j \leftarrow v_j\}_{j < i}}^{\Gamma}) \wedge r\{x_i \setminus v_i\}_{i \leq n} \{x \setminus v\} \rho \rightarrow_{\lambda}^* \mathbf{True}\} \\
\llbracket \{x:B \mid r\} \rrbracket_{\rho}^{\Gamma} &\doteq \{v \mid v \in \llbracket B \rrbracket_{\rho}^{\Gamma} \wedge r\{x \setminus v\} \rho \rightarrow_{\lambda}^* \mathbf{True}\} \\
\llbracket x:R_x \rightarrow R \rrbracket_{\rho}^{\Gamma} &\doteq \{v \mid \lfloor \Gamma \rfloor_{H\lambda} \vdash_{\mathcal{L}C} v : \lfloor R_x \rfloor_{H\lambda} \rightarrow \lfloor R \rfloor_{H\lambda} \\
&\quad \wedge \forall v_x \in \llbracket R_x \rrbracket_{\rho}^{\Gamma}. v \ v_x \rightarrow_{\lambda}^* v' \ \text{s.t.} \ v' \in \llbracket R \rrbracket_{\rho\{x \leftarrow v_x\}}^{\Gamma}\}
\end{aligned}$$

Definition C.9 (Interpretation of Rocq types).

$$\begin{aligned}
\llbracket \{x:\mathbb{U} \mid Q\} \rrbracket_{\rho}^{\Gamma} &\doteq \{v \mid \lfloor \Gamma \rfloor_{H\lambda} \vdash_{\mathcal{L}C} v : \lfloor \mathbb{U} \rfloor_{Q\lambda} \wedge \lfloor Q \rfloor_{Q\lambda} \{x \setminus v\} \rho \rightarrow_{\lambda}^* \mathbf{True}\} \\
\llbracket \forall(x:\$, \$) \rrbracket_{\rho}^{\Gamma} &\doteq \{v \mid \lfloor \Gamma \rfloor_{H\lambda} \vdash_{\mathcal{L}C} v : \lfloor \$ \rfloor_{Q\lambda} \rightarrow \lfloor \$ \rfloor_{Q\lambda} \\
&\quad \wedge \forall v_x \in \llbracket \$ \rfloor_{\rho}^{\Gamma}. v \ v_x \rightarrow_{\lambda}^* v' \ \text{s.t.} \ v' \in \llbracket \$ \rrbracket_{\rho\{x \leftarrow v_x\}}^{\Gamma}\}
\end{aligned}$$

$$\llbracket \text{Pack}(\forall(x_i : \mathbb{S}_i)_{i \leq n}, \mathbb{S}) \rrbracket_\rho^\Gamma \doteq \llbracket \forall(x_i : \mathbb{S}_i)_{i \leq n}, \mathbb{S} \rrbracket_\rho^\Gamma$$

LEMMA C.10. For all refinement r , $r = \lfloor r \rfloor_{H\lambda} = \lfloor \text{RtoP}(r) \rfloor_{Q\lambda}$.

PROOF. The first equality is by definition. For the second, notice that $\lfloor \text{RtoP}(r) \rfloor_{Q\lambda}$ does not contain a call to wf_{TC} . It is easy to prove that $\lfloor \text{RtoP}(r) \rfloor_{Q\lambda}$ is equal to r where the outmost applications of functions, operators and top-level constants have been replaced by the corresponding fresh variable in RV. Let $(\mathbb{Q}_1, \dots, \mathbb{Q}_n) \doteq \text{Hyp}(r)$ and for each i ,

$$\mathbb{Q}_i \doteq \forall z_i, (f_i \text{_rel} / f_i \text{.(rel)} / f_i \text{.(rel_u)}) \overline{\text{RtoU}(r_i)} z_i$$

Then, we have

$$\begin{aligned} \lfloor \text{RtoP}(r) \rfloor_{Q\lambda} &= \lfloor \mathbb{Q}_1 \rightarrow \dots \rightarrow \mathbb{Q}_n \rightarrow \mathbf{is_true}(\text{RtoU}(r)) \rfloor_{Q\lambda} \\ &= \lfloor \text{RtoU}(r) \rfloor_{Q\lambda} \{z_i \setminus f_i \overline{\lfloor \text{RtoU}(r_i) \rfloor_{Q\lambda}}\}_{i \leq n} \\ &= r \end{aligned} \quad \square$$

LEMMA C.11. Let $v \in \llbracket B \rrbracket_\rho^\Gamma$. Then, $\lfloor \Gamma \rfloor_{H\lambda} \vdash_{\mathcal{L}C} v : B$.

PROOF. By structural induction on B . The case B is builtin is direct by definition of the models. If $B = TC$, then $v = C v_1 \dots v_n$, we have $\Gamma(TC)(C) = (x_i : \{x_i : B_i \mid r_i\})_{i \leq n} \rightarrow \{x : TC \mid r\}$. Let $\rho_i \doteq \rho \{x_j \leftarrow v_j\}_{j < i}$. By hypothesis, for each $i \leq n$, we have $v_i \in \llbracket R_i \rrbracket_{\rho_i}^\Gamma$ which implies by induction that $\lfloor \Gamma \rfloor_{H\lambda} \vdash_{\mathcal{L}C} v_i : B_i$. Thus, by applications and since $\lfloor \Gamma(TC)(C) \rfloor_{H\lambda} = B_1 \rightarrow \dots \rightarrow B_n \rightarrow TC$, we derive $\lfloor \Gamma \rfloor_{H\lambda} \vdash_{\mathcal{L}C} v : TC$. $\square$

THEOREM 4.2. Let R be a refinement type in λ_r and Γ a λ_r context of datatypes such that R is well-formed under Γ . For any assignment ρ , $\llbracket R \rrbracket_\rho^\Gamma = \llbracket \text{TtoR}(R) \rrbracket_\rho^\Gamma$.

PROOF. By induction on the structure of the type. First, let $R \doteq \{x : B \mid r_x\}$. If B is a builtin, we are done because $\lfloor \mathbb{Q} \rfloor_{Q\lambda} = \lfloor \text{RtoP}(r_x) \rfloor_{Q\lambda} = r_x$ (by lemma C.10) and $v \in \llbracket B \rrbracket_\rho^\Gamma$ is clearly equivalent to $\lfloor \Gamma \rfloor_{H\lambda} \vdash_{\mathcal{L}C} v : B$.

Let $B \doteq TC$ and thus $\text{TtoU}(R) = \{x : TC_u \mid \text{wf}_{TC} x \wedge \text{RtoP}(r_x)\}$. By lemma C.10, we have $\lfloor \text{RtoP}(r_x) \rfloor_{Q\lambda} = r_x$, so on both sides we have $r_x \{x \setminus v\} \rightarrow_\lambda^* \mathbf{True}$.

- $v \in \llbracket R \rrbracket_\rho^\Gamma \implies v \in \llbracket \text{TtoU}(R) \rrbracket_\rho^\Gamma$. By definition, v is of the shape $C v_1 \dots v_n$. From lemma C.11, we have $\lfloor \Gamma \rfloor_{H\lambda} \vdash_{\mathcal{L}C} v : TC$. We still need to prove

$$(\forall i \in [1..n]. v_i \in \llbracket R_i \rrbracket_{\rho_i}^\Gamma) \wedge r_C \sigma \rho \rightarrow_\lambda^* \mathbf{True} \implies \lfloor \text{wf}_{TC} x \rfloor_{Q\lambda} \{x \setminus v\} \rho \rightarrow_\lambda^* \mathbf{True}$$

Where for each i , $\rho_i = \rho \{x_j \leftarrow v_j\}_{j < i}$, where $\sigma = \{x_i \setminus v_i\}_{i \leq n} \{x \setminus v\}$, and with $\Gamma(TC)(C) = (x_i : R_i)_{i \leq n} \rightarrow \{x : TC \mid r_C\}$.

Let $R_i \doteq \{x'_i : B_i \mid r_i\}$ for each $i \leq n$ (there are no higher-order arguments of data constructors in our system). By definition,

$$\lfloor \text{wf}_{TC} x \rfloor_{Q\lambda} \{x \setminus v\} \rightarrow_\lambda^* \lfloor \text{RtoP}(r_C) \rfloor_{Q\lambda} \sigma \wedge \bigwedge_{i \leq n} \lfloor \mathbb{Q}_i \{x'_i \setminus x_i\} \rfloor_{Q\lambda} \sigma$$

where $\mathbb{Q}_i$ is such that $\text{TtoU}(R_i) = \{x'_i : \text{Prim}(B_i) \mid \mathbb{Q}_i\}$. By lemma C.10, $\lfloor \text{RtoP}(r_C) \rfloor_{Q\lambda} = r_C$, and therefore $\lfloor \text{RtoP}(r_C) \rfloor_{Q\lambda} \sigma \rho \rightarrow_\lambda^* \mathbf{True}$. For each $i \leq n$, we obtain by induction hypothesis $v_i \in \llbracket R_i \rrbracket_{\rho_i}^\Gamma$, and thus in particular $\lfloor \mathbb{Q}_i \{x'_i \setminus x_i\} \rfloor_{Q\lambda} \rho_i \rightarrow_\lambda^* \mathbf{True}$ and we conclude since $\text{fv}(v_i) \cap \text{dom}(\sigma) = \text{dom}(\rho_i)$.

- $v \in \llbracket \text{TtoU}(R) \rrbracket_\rho^\Gamma \implies v \in \llbracket R \rrbracket_\rho^\Gamma$. By hypothesis, we have $\lfloor \Gamma \rfloor_{Q\lambda} \vdash_{\mathcal{L}C} v : TC$ and $\lfloor \text{wf}_{TC} x \rfloor_{Q\lambda} \{x \setminus v\} \rho \rightarrow_\lambda^* \mathbf{True}$. Since we have a derivation for v of type TC , v can only be of the shape $C v_1 \dots v_n$, and there must be $(R_i)_{i \leq n}$, x and r_C such that $\Gamma(TC)(C) = (x_i : R_i)_{i \leq n} \rightarrow \{x : TC \mid r_C\}$. We need to prove: (1) $r_C \sigma \rho \rightarrow_\lambda^* \mathbf{True}$ and (2) $\forall i \in [1..n]. v_i \in \llbracket R_i \rrbracket_{\rho_i}^\Gamma$.

For (1), $\llbracket \text{wf}_{TC} x \rrbracket_{Q\lambda} \{x \setminus v\} \rho \rightarrow_{\lambda}^* \mathbf{True}$ implies as before $\llbracket \text{RtoP}(r_C) \rrbracket_{Q\lambda} \sigma \rho = r_C \sigma \rho \rightarrow_{\lambda}^* \mathbf{True}$ by lemma C.10. For (2), let $i \leq n$. Then R_i is a refinement of a base type by restriction of the system, let $R_i \doteq \{x'_i : B_i \mid r_i\}$ and $\text{TtoU}(R_i) \doteq \{x'_i : \text{Prim}(B_i) \mid Q_i\}$. Again, $\llbracket \text{wf}_{TC} x \rrbracket_{Q\lambda} \{x \setminus v\} \rho \rightarrow_{\lambda}^* \mathbf{True}$ implies $\llbracket Q_i \rrbracket_{Q\lambda} \sigma \rho = r_C \sigma \rho \rightarrow_{\lambda}^* \mathbf{True}$ by lemma C.11. Moreover, since there is a derivation for v , there is also a derivation for v_i . Hence, $v_i \in \llbracket R_i \rrbracket_{\rho}^{\Gamma}$, and thus by induction $v_i \in \llbracket R_i \rrbracket_{\rho}^{\Gamma}$.

Now, consider $R = x : R_x \rightarrow R'$. If R_x is an arrow type, it is translated to an instance of `Pack`. The translations of `Pack` and dependent arrows of Rocq and of λ_r in $\lambda_{\rightarrow}$ are all equal. So we conclude by induction hypothesis, since the interpretations are built in the same way. $\square$

D Implementation of Pack

Our description of `Packs` in the main text was simplified to records containing the refined definition, graph relation, the lemma relating them and functionhood for the graph relation. In reality, to be able to implement a generic `Pack` typeclass for all function arities and for higher-order arguments (becoming nested `Packs`), we encode function arguments as a list. In this appendix, we give more details about the implementation of `Packs`, and precise the translations.

D.1 Definition of Function Packs

The argument types of a function are stored in a dependent list, the type `ArgListT` is the type of dependent lists of argument types. The type `ArgList` represents a dependent list of terms of those types given by a term `argTps : ArgListT`. Analogously, the types `UArgListT` and `UArgList` represent respectively lists of unrefined types, and lists of arguments of a term `uargTps : UArgListT`. The relation `projectsArgListT` indicates whether an `UArgListT` contains exactly the unrefined types of a given `ArgListT` and the function `prArgList` extracts an `UArgList` out of an `ArgList`.

We use these encodings of arrow types to define the typeclasses `Pack` (refined) and `uPack` (unrefined) in Rocq.

Definition D.1 (Function Packs). In the parameters, `argTps` is the dependent list of the refined argument types, `uargsTps` the list of unrefined argument types, related to `argTps` by the projection `prj`. The unrefined return type is `T`, refined by the predicate `p`.

```
Class Pack (argTps:ArgListT) {uargTps:UArgListT} {prj:projectsArgListT argTps uargTps}
(T: Type) (p: ∀ (args: ArgList argTps), T → Prop) := {
  def (args:ArgList argTps): {v:T | p args v}; (* Refined definition *)
  rel (uargs: UArgList uargTps) (v:T): Prop; (* Graph relation *)
  f_f_rel (args:ArgList argTps) (v:T): [def args] = v ↔ rel (prArgList args uargTps prj) v;
  funct (uargs: UArgList uargTps) v v': rel uargs v → rel uargs v' → v = v' }.
```

```
Class uPack (uargTps: UArgListT) (T: Type) := {
  rel_u (uargs: UArgList uargTps): T → Prop; (* Graph relation *)
  funct_u (uargs:UArgList uargTps) (v v':T): rel_u uargs v → rel_u uargs v' → v = v' }.
```

Because of the encoding, we use a tactic `buildPackG` to populate the typeclass instances and tactics to project individual fields. `buildPackG` takes the refined definition, the unrefined graph relation, and the two `Pack` lemmas as parameters. There is a similar tactic `buildUPackG` for an unrefined `Pack`. The field of the refined function is retrieved with the tactic `getPackF` implicitly doing the conversion from the encoding. The graph relation is retrieved with `getPackRel` from a refined `Pack` and `getUPackRel` from an unrefined `Pack`. There are also tactics to retrieve the lemmas.

The projection of a refined Pack x to an unrefined Pack can still be defined as $\text{projPack}(x) \doteq \{|\text{rel_u} := x.\text{rel}; \text{funct_u} := x.\text{pack}.\text{funct}|\}$, despite the encoding.

Translations. With the definition of Packs as using lists of arguments, the translations of function types as Pack are changed. In the unrefined translation, we have the following, using the list notation for an element of type UArgListT .

$$\text{TtoU}((x_i:R_i)_{i \leq n} \rightarrow R) \doteq \text{uPack} [\text{TtoU}(R_1), \dots, \text{TtoU}(R_n)] \text{TtoU}(R)$$

In the refined translation, we have:

$$\begin{aligned} &\text{TtoR}((x_i:R_i)_{i \leq n} \rightarrow R) \\ &\quad \doteq \text{Pack} [x_1:\text{TtoR}(R_1), \dots, \text{TtoR}(R_n)] \mathbb{U} (\lambda \text{args } v, \text{ltac:} (\text{flattenP } p \text{ args } v)) \end{aligned}$$

Where $\{x:\mathbb{U} \mid p \ x_1 \dots x_n \ x\} = \text{TtoR}(R)$ and the first argument of Pack is a dependent list of type ArgList . We use a tactic `flattenP` to transform the $n + 1$ -ary predicate p into a predicate that takes an argument list of length n as first parameter.

Example D.2. Consider the type R of the function `apply` from the overview (§ 2.6).

$$R \doteq f:(ys:\text{IList} \rightarrow \{v:\text{Int} \mid 0 \leq v\}) \rightarrow xs:\text{IList} \rightarrow \{v:\text{Int} \mid v = f \ xs\}$$

The type of the higher-order parameter f is translated as:

$$\mathbb{S}_f \doteq \text{Pack}[\text{IList}] \mathbb{Z} (\lambda \text{args } v, \text{match args with } [ys] => \text{leqZ_rel } 0 \ v \ \text{true} \ \text{end})$$

And the full translation of R as the type of the Rocq refined function is thus:

$$\forall (f : \mathbb{S}_f) (xs : \text{IList}), \{v:\mathbb{Z} \mid \forall z, (\text{getPackRel } f) \ [xs] \ z \rightarrow v == z\}$$

D.2 Subsumption for Packs

In this section, we describe subsumption for Packs. In order to uniformly translate subsumption casts of subset types and of function Packs inside the list of arguments of a function, we define a typeclass for subsumption casts. This class has two fields: the cast result and a property that cast does not affect projection (since subtyping only affects refinements).

```
Class Subsumption {U: Type} (S T : Type) {prS: Projection S U} {prT: Projection T U} (x: S) :=
{ sub_res: T; cast_pr: [sub_res] = [x] }.
```

The instance `subRef` for subset types is defined using the subsumption function given in § 3.5.2.

To define an instance for Packs, we first define an inductive type to record the requirements for a successful subsumption cast between ArgLists :

```
Inductive SubArgList: ArgListT → ArgListT → Type :=
| noArgsSub: SubArgList noArgsT noArgsT
| consArgsSub (U S T: Type) (prS: Projection S U) (prT: Projection T U)
  (argsS: ∀(x:S), ArgListT) (argsT: ∀(x:T), ArgListT) (cast: ∀(x:S), Subsumption S T x) (x:S)
  : SubArgList (argsS x) (argsT (cast x)) (sub_res) → SubArgList (S ::RT argsS) (T ::RT argsT).
```

Where `noArgsT` and `::RT` are the constructor for ArgListT .

The instance `subPack` for Packs has the signature below. Its definition is technical but follows from the signature.

```
Instance subPack {argTps: ArgListT} {T: Type} {p: ∀ x:ArgList argTps, T → Prop}
{argTps': ArgListT} {q: ∀(x:ArgList argTps'), T → Prop} {uargTps: UArgListT}
{z: projectsArgListT argTps uargTps} {z': projectsArgListT argTps' uargTps}
(fpack: \fClass{} argTps uargTps z T p) (castArg: SubArgList argTps' argTps)
(castT: ∀ (args:ArgList argTps) (args':ArgList argTps'),
```

```

Subsumption {v:T | p args v} {v:T | q args' v} (fpack.(f) (subArgList castArg args)))
: Subsumption (Pack argTps uargTps z T p) (Pack argTps' uargTps z' T q) fpack.

```

E Detailed Explanations of Proof Automation

E.1 Full Explanation of Lemmas Relating Functions with their Graph Relations

In § 3.3.2, we described part of the translation of a reflected function f : deriving the refined translation using tactics, the unrefined graph relation, and a lemma f_rel to relate both. On top of that, Lava generates numerous other lemmas that are required both to prove f_rel and for reasoning steps in automation tactics.

The lemmas generated are, in order:

- (1) the *functionhood lemma* f_rel_funct stating functionhood of the relation f_rel , that is, the relation relates a single output to a set of inputs;
- (2) the *existence lemma* f_rel_ex stating that the projection of the result of applying the function f lies in the image of its relation f_rel ;
- (3) the theorem f_rel relating the function f with its relation;
- (4) the *instantiating lemma* f_rel_mk that takes terms $\bar{t}_i$ and proof terms stating that the $\bar{t}_i$ satisfy the refinements of the argument types of f and constructs a terms v satisfying $f_rel \bar{t}_i v$;

Besides the these four lemmas, the translation also generates a series of *inversion lemmas* for the graph relation. These lemmas state the equivalence of an application of f_rel with a different condition that is easier to prove. For example, the lemma for the successor case of the relation for multiplication of Peano natural numbers states $\forall (n : Nat) (v : Nat) (m : Nat) : mult_rel (Suc m) n v \leftrightarrow \exists (v' : Nat) (v_def : add_rel n v' v), mult_rel m n v'$. These lemmas are applied for backwards reasoning in the goal in the custom tactics, typically followed by instantiating the existential variables created by the lemmas.

The existence lemma is by far the hardest of these lemmas, for one simple reason: it is the only lemma that actually has to reason about the function definition itself (all other lemmas are either trivial corollaries or avoid reasoning about the functions (possibly utilizing the rewriting lemma for this purpose)).

E.2 Full Explanation of the Solver Tactic

As mentioned already in § 5 the solver tactic is build on four levels of simplification and solving tactics of increasing strength.

We will now describe these simplification and goal solving tactics in more detail.

The fast_done tactic. The `fast_done` tactic combines several native Coq tactic to quickly solve a goal (or fail quickly).

The quick_simpl and quick_solve tactics. The `quick_cleanup` tactic contains several (if-then-else related) goal rewrite steps, `autounfold` and `autorewrites` with some hint databases. In particular, it will simplify projections of casts and fully applied refined binary operators. The `shape_based` tactic is defined as a big match (>100 branches) on the proof state doing a single simple forward reasoning step in a hypothesis or goal. It only makes reasoning steps that cannot interfere with our other automation tactics (i.e. they make proving the goal strictly easier) and they are all very simple and fast. Many of the branches simply rewrite with a number of lemmas regarding booleans, equality and boolean connectives. This tactic also serves as the main "plumbing" (along with a number of rewrite/unfold/resolve hints added to a number of hint databases which are applied automatically in many of the automation tactics described in this section) for the overall automation machinery bringing the proof state into a form the other automation tactics can (more easily) handle.

Fig. 10. Simplified definition of the fast_done tactic.

```

Ltac fast_done :=
solve [
quick_wff_wit (* automatically apply the wff lemmas *)
| unshelve eassumption
| symmetry; unshelve eassumption
| reflexivity (* works both for Leibniz equality and the translated equality *)
| destruct_unit (* destruct unit terms, useful to prove they are all equal *)
| lia (* solve integer arithmetic goals *)
| congruence (* solve goals in theory of equality and uninterpreted functions *)
| unshelve intuition (* simplify and apply tauto,
                        a decision procedure for constructive propositional logic *)
].
Ltac fast_solver := fast_done.

```

```

Ltac quick_simpl := try quick_cleanup; repeat shape_based.

Ltac solving_loop := first [
fast_solver
| f_equal_ind
| progress shape_based
| congruence
| progress subst
| progress (simpl in *)
| progress autorewrite with lia_rewrites fix_notation_hints in *
| left; simpl_loop
| right; simpl_loop
].

Ltac quick_solve := solve [ quick_wff_wit | quick_simpl; repeat solving_loop ].

```

Fig. 11. Simplified definition of quicksolve tactic.

The `f_equal_ind` tactic applies the `f_equal` tactic to goals that are equalities between applications of the same constructor of an inductive data type. It is useful in various other automation tactics involving backwards reasoning, in particular for reasoning about the automatically derived equality operators for inductive data types. The `solving_loop` tactic is the backbone of the `quick_solve` tactic. It corresponds to a single goal solving or simplification step. The `quick_solve` tactic is then defined as (basically) running first `quick_simpl` and then running `solving_loop` until the goal is solved. We explicitly try solving the goal with `quick_wff_wit` first, to produce faster and simpler proofs for simple well-formedness goals.

The cleanup and quick_solve tactics. The `unpack_all` tactic destructs context variables of type `Pack_` or `uPack_`.

The `axiomatize_ih_specializations` tactic replaces the projections of the applications of induction hypothesis with free variables of same type and adds a witnesses of the refinement of that


```

Ltac quick_simple_cleanup_steps :=
try unify_vars; try (specialize_hyps; try unify_vars).

Ltac initial_simple_cleanup_steps :=
quick_simpl;
repeat progress autounfold with lia_unfold in *;
simpl_proj;
try unpack_all;
try (axiomatize_ih_specializations; quick_simpl);
try (axiomatize_terms; quick_simpl);
quick_simple_cleanup_steps.

Ltac inversion_cleanup :=
(* forward reasoning *)
try quick_simple_cleanup_steps;
(* backwards reasoning *)
(* backwards reasoning in hypotheses *)
try (invert_axiomatizations; quick_simple_cleanup_steps;
      try instantiate_hyps; try specialize_hyps);
(* backwards reasoning in goal *)
try (autorewrite with f_rel_back; try instantiate_goals);
quick_simple_cleanup_steps.

Ltac cleanup_recurse :=
quick_simpl;
try initial_simple_cleanup_steps;
try inversion_cleanup

Ltac cleanup :=
quick_simpl;
saturate_context;
try cleanup_recurse;
saturate_context;
repeat cleanup_recurse.

Ltac finish := solve [
quick_wff_wit
| first [quicksolve | repeat split; quicksolve] | timeout 5 verit ].

Ltac oracle := solve [
quick_wff_wit
| quicksolve
| cleanup; finish].

```

Fig. 12. Simplified definition of the oracle tactic

application of the induction hypothesis to the context. It also unifies applications of an induction hypothesis that differ only in witnesses (using proof irrelevance). This helps remove complicated proof terms from the proof state and unify applications of induction hypothesis that differ only in refinement witnesses.

The `axiomatize_terms` tactic replaces the projections of full applications of reflected functions with variables axiomatized as result of the corresponding relations (and similarly for projections of applications of functions extracted from Packs). A refinement witness for the application is also added to the context.

The `unify_vars` tactic "unifies" variables axiomatized as the results of the same applications of graph relation using the functionhood lemma of the relation. It also removes duplicated assumptions from the context and substitutes variables by equalities in the context if possible.

The `specialize_hyps` tactic specialize uniquely axiomatized antecedents in hypothesis (result variables of graph relations that are given elsewhere in the context). It also tries to specialize hypothesis with proof terms generated by the `quick_wff_wit` or `quick_solve` tactics.

The `invert_axiomatizations` tactic inverts the hypothesis axiomatizing variables using the relations, whenever this inversion results in no additional subgoals.

The `instantiate_hyps` tactic creates terms for uniquely axiomatized variables in hypotheses satisfying the axiomatization. For this, we transitively traverse the relations and create the refined term (uniquely) axiomatized as that variable, then we replace the projection of that refined term with a variable axiomatized using a graph relation. Finally we specialize the hypothesis with it that variable and with it's defining axiom.

`autorewrite with f_rel_back` rewrites goals using the backwards reasoning lemmas.

The `instantiate_goals` tactic creates (if not already present in the context) terms to specialize the free (existentially quantified) variables in the goal with, then specializes the goal with them (using the `exists` tactic).

The `saturate_context` tactic generates refinement witnesses for the variables axiomatized using graph relations in the context. This requires reconstructing refined terms for result variables of graph relations (similar to what is done in the `instantiate_hyps` tactic). As a consequence, it is frequently very slow, so in the actual tactic definitions, it is only run after a full (failed) run without it. As a consequence, in the few cases where context saturation is actually necessary to prove a goal, the verification time for the goal will be quite substantial. Indeed, the final proof obligation from the Overview example mention in § 6 that takes up a significant fraction of the total verification time for our benchmarks takes so long to prove mostly because it requires context saturation.

The solver tactic. The `lia_preprocessor` tactic autounfolds and autorewrites with a number of lemmas to simplify integer reasoning goals to something the `lia` tactic can hopefully handle. It also tries to decide which branch of a disjunction in the goal to take using the `oracle` tactic to prove subgoals and it tries to specialize hypotheses with proof terms using the `oracle` tactic. The `solver` tactic then improves upon the `oracle` tactic merely by additionally preprocessing using the `lia_preprocessor` tactic and by shelving subgoals it cannot solve.

```

Ltac preprocessor :=
repeat progress (try cleanup; try lia_preprocessor).
Ltac solver := first [
quick_wff_wit
| lia_preprocessor; lia
| preprocessor;
first [lia | oracle | shelve]
].

Ltac solver := first [
quick_wff_wit
| quicksolve
| quick_simpl; repeat progress first [cleanup | lia_preprocessor];
first [quicksolve | solve [unshelve verit] | shelve]
].

```

Fig. 13. Simplified explanation of solver tactic.